

Lifecycle Cost Redistribution in Workflow Automation: An Exploratory Benchmark of Code, Visual, and AI Paradigms

Rohit R

rohit.r@pes.edu

PES University

Department of Computer Science (AI & ML)
Bengaluru, Karnataka, India

Table 1: Notation used throughout the paper.

Symbol	Meaning
IC	Implementation Cost
MC	Maintenance Cost
RC	Recovery Cost
VC	Verification Cost
TWC	Total Workflow Cost ($= IC + MC + RC + VC$)
LCRI	Lifecycle Cost Redistribution Index
CDH	Complexity Displacement Hypothesis
ACE	Abstraction Compression Effect

Abstract

Recovery cost emerged as the primary differentiator between workflow paradigms under coordination-demanding tasks, yet remains unmeasured in prior comparative evaluations. This paper presents a controlled empirical benchmark evaluating three workflow paradigms—Code-Based (Python), Visual Workflow (n8n), and AI-Generated (zero-shot GPT-4 Turbo)—across four tasks of escalating coordination demands. We operationalize four time-based metrics—Implementation Cost (IC), Maintenance Cost (MC), Recovery Cost (RC), and Verification Cost (VC)—to assess Total Workflow Cost (TWC). AI prompting reduces implementation cost relative to manual coding, but this saving is associated with increased downstream effort: under multi-agent orchestration, the visual builder exhibits a 20.4× recovery-cost escalation, whereas AI shifts effort from implementation toward debugging and maintenance. These patterns are consistent with lifecycle cost redistribution, an interpretation formalized as the Complexity Displacement Hypothesis.

CCS Concepts

• **Software and its engineering** → **Software creation and management; Integrated and visual development environments; Empirical software validation.**

Keywords

Workflow automation, complexity displacement, low-code, visual programming, LLM code generation, empirical benchmark, software maintainability, coordination complexity

1 Introduction

Modern software engineering increasingly relies on high-level frameworks to execute data pipelines, orchestrate microservices, and coordinate business logic [4, 8, 18, 19]. Three widely adopted paradigms have emerged: (i) traditional code-based scripting in general-purpose languages such as Python; (ii) visual pipeline builders exemplified by n8n, Zapier, and Make, which allow developers to construct pipelines through drag-and-drop node graphs; and (iii) AI-generated systems, where a natural-language prompt to a large language model produces either executable code or a pipeline graph [20–22].

Prior comparative analyses of these paradigms focus almost exclusively on development velocity [3, 5]. This narrow focus obscures critical lifecycle trade-offs: a workflow that is rapid to implement may prove difficult to verify, expensive to recover from failures, or costly to modify as requirements evolve [16, 17]. Systematic evaluations of end-to-end lifecycle costs across multiple abstraction levels remain scarce. Without such evaluations, organizations lack evidence for selecting paradigms suited to their operational profiles—particularly for coordination-demanding workflows where post-implementation costs dominate.

We address this gap through a controlled benchmark that measures lifecycle cost distributions across three workflow paradigms. Our primary research objective is to determine whether abstraction shifts effort between phases rather than eliminating it, and which cost components are most affected. We designed four tasks spanning a sequential pipeline, a fault-tolerant retry chain, a conditional decision tree, and a multi-agent orchestration, implemented each across Python (code), n8n (visual), and GPT-4 Turbo (AI), and measured four time-based metrics (IC , MC , RC , VC) to compose the Total Workflow Cost (TWC). The Complexity Displacement Hypothesis (CDH) serves as the interpretive lens for analyzing these measurements, but the benchmark data stand independently: the cost distributions across tasks and paradigms are empirical findings that any theory of workflow automation must accommodate. CDH is an interpretive framework applied to the benchmark observations rather than a validated predictive theory; it generates directional predictions (effort redistribution under abstraction) that this benchmark can inform but cannot confirm.

The contributions are three-fold:

- (1) **A reproducible lifecycle benchmark.** We present an open-source benchmark suite of four workflow tasks and operationalize four repeatable time-based metrics (IC , MC , RC , VC). We also propose the Lifecycle Cost Redistribution

Index (LCRI)—a portable summary statistic capturing the fraction of lifecycle cost occurring after implementation—as an exploratory descriptive metric for future cross-paradigm comparisons. The benchmark dataset provides empirical constraints independent of any particular theoretical lens.

- (2) **Recovery cost as the dominant signal.** The largest observed redistribution is that recovery cost scales disproportionately under coordination-demanding workflows for non-code paradigms. Visual builders exhibit a 20.4× escalation and AI shows a 4.9× escalation under multi-agent orchestration, while code-based recovery remains flat (19–23 s). This divergence—consistent across all four tasks—indicates that recovery behavior emerged as the most pronounced separation in this benchmark, a dimension absent from prior single-metric evaluations.
- (3) **Empirical observations of lifecycle cost redistribution.** AI prompting reduces implementation cost relative to manual scripting, but this saving is offset by elevated post-implementation costs. Visual tools exhibit a 20.4× recovery-cost escalation under orchestration, and prompt-driven systems redistribute effort toward debugging and maintenance—each pattern reflects displacement of effort, though alternative explanations (platform maturity, evaluator proficiency, task selection) cannot be ruled out. These observations do not contradict the Complexity Displacement Hypothesis, one of several possible interpretive lenses discussed in Section 6.3.

Our conceptual framework follows a clear hierarchy, designed to keep the benchmark data separable from any particular interpretation. The raw measurements (*IC*, *MC*, *RC*, *VC*) are the primary contribution. LCRI is a descriptive aggregation of those measurements. CDH and ACE are analytical lenses—one possible interpretation of the observed patterns, not validated theories. Additional derived measures (*RCAF*, *ICR*, *COS*, *ACR*) are defined where used in the methodology. The benchmark data would remain informative even if CDH and ACE are ultimately rejected or refined, because the cost distributions across four tasks and three paradigms provide independent empirical constraints that any theory of workflow automation must accommodate.

The paper is organized as follows. Section 2 situates our work within the literature on low-code platforms, software maintainability, and AI-assisted development. Section 3 presents the cost decomposition framework. Section 4 describes the benchmark design and experimental methodology. Section 5 reports the results. Section 6 discusses the findings and interprets them through the Complexity Displacement Hypothesis. Section 7 addresses threats to validity. Section 8 concludes and outlines future work.

2 Related Work

Our work bridges three research strands: low-code and visual workflow platforms, software engineering metrics for maintainability, and AI-assisted code and workflow generation.

2.1 Low-Code and Visual Platforms

Lethbridge [1] observed that low-code platforms often inherit the structural complexity of traditional programming, noting

that “low-code is often high-code.” Luo *et al.* [2] conducted a practitioner survey identifying configuration management, debugging, and scalability as the principal challenges in low-code development. Pinho *et al.* [3] systematically reviewed usability in low-code platforms and found that while authoring efficiency improves, cognitive load in debugging complex workflows remains a significant concern.

Washburn *et al.* [4] empirically evaluated developer experience on visual graph-based no-code platforms and reported that while simple workflows benefit from visual representation, the advantage degrades as node count and branching depth increase. Rokis and Kirikova [5] identified enterprise scalability challenges in model-driven visual environments, noting that the visual canvas becomes a liability when workflows exceed approximately twelve nodes. Vashist and Singh [6] compared Robotic Process Automation with low-code platforms and found that error recovery is substantially slower in visual paradigms. Barkadir *et al.* [7] proposed formal verification methods for workflow pipelines, although their approach assumes a single paradigm and does not compare abstraction levels.

These studies collectively establish that visual workflows reduce authoring friction but introduce workflow burden at scale. Our work extends this literature by providing a direct, metric-based comparison of code, visual, and AI-generated paradigms under identical task conditions.

Table 2 summarizes how prior work relates to our study across six dimensions. These dimensions were selected because they correspond directly to the lifecycle-cost constructs evaluated in this study: multi-paradigm comparison, full lifecycle measurement, coordination-demand control, identical-task design, time-based metrics, and theory-driven prediction testing. No prior study simultaneously addresses all six.

Table 2: Condensed research gap matrix positioning this work relative to prior empirical studies. (• = addressed by study, ◦ = partially addressed, – = not addressed.)

Study	Multi-Paradigm	Full Lifecycle	Coord. Compl.	Same Task	Time-Based	Theory-Driven
Lethbridge [1]	–	–	–	–	–	–
Luo <i>et al.</i> [2]	•	–	–	–	–	–
Pinho <i>et al.</i> [3]	•	–	–	–	–	–
Washburn <i>et al.</i> [4]	•	–	–	•	•	–
Rokis and Kirikova [5]	•	–	–	–	–	–
Zhang <i>et al.</i> [19]	•	–	–	•	•	–
Martini <i>et al.</i> [16]	–	•	–	–	•	–
Strandberg [17]	–	•	–	–	–	–
Jones and Miller [13]	•	–	–	•	•	–
This work	•	•	•	•	•	•

Not all evidence points in the same direction. Several industry reports and controlled studies document sustained productivity improvements from low-code and AI-assisted development that persist beyond initial implementation [3, 5]. Strandberg [17] observed that enterprise automation teams using visual tools reported downstream costs remaining proportional rather than escalating in longitudinal tracking. More recently, Turner *et al.* [25] argued that AI

code generation reduces debugging effort when integrated with structured verification pipelines. These findings do not directly contradict CDH—they may reflect different task domains or organizational contexts—but they establish that the displacement pattern observed in our benchmark is not a universal property of abstraction.

2.2 Software Maintainability Metrics

The theoretical basis for our metric framework draws from classical software engineering. McCabe [10] introduced cyclomatic complexity as a measure of control-flow complexity; Chidamber and Kemerer [9] extended these concepts to object-oriented environments via the CK metrics suite. Fenton and Bieman [8] provided a comprehensive treatment of software metrics, emphasizing that no single measure captures lifecycle cost. Weyuker [11] established formal properties for evaluating complexity measures, many of which inform our choice of multi-dimensional metrics.

Recent empirical work has examined debugging and recovery. Silva *et al.* [12] investigated whether modern automated debugging tools reduce troubleshooting time and found that context-switching between tools introduces overhead. Jones and Miller [13] quantified cognitive load in visual versus textual debugging and concluded that visual debugging imposes higher mental rotation costs as branching increases. Alshakarji *et al.* [14] used fMRI to study how developers understand automated workflows, finding that non-linear execution paths significantly increase neural activation in regions associated with working memory.

Garcia and Escalona [15] proposed quality assurance metrics specific to low-code environments, while Martini *et al.* [16] quantified the long-term interest of structural technical debt. Our work operationalizes these conceptual frameworks into four directly measurable time-based metrics tailored for cross-paradigm comparison.

2.3 AI-Assisted Code and Workflow Generation

The rapid advancement of large language models has spawned a growing body of work on AI-assisted code generation. Chen *et al.* [27] introduced Codex and evaluated its performance on code synthesis tasks. Sandoval Alcocer *et al.* [22] characterized the shift from code production to code orchestration enabled by AI coding assistants. Xu *et al.* [21] conducted a systematic literature review of generative AI in software engineering, identifying verification and trust as the primary open challenges.

Wang *et al.* [20] described the “testing paradox”: generative AI increases the importance of testing while itself failing at automated test creation. Liang [23] demonstrated that RLHF-based training can introduce self-admitted technical debt in LLM-generated code. Zhang *et al.* [19] compared low-code programming with and without LLM support, finding that while LLMs accelerate initial drafts, they do not reduce debugging effort.

Saló [24] evaluated agentic frameworks for multi-agent software engineering and reported instability in long dependency chains. Turner *et al.* [25] examined the verifiability of AI-generated code and proposed accessibility-focused evaluation criteria. Lopez [26] provided a comprehensive taxonomy of LLM

benchmarks in software engineering, situating code generation within a broader evaluation landscape.

Our work differs from these studies in two important respects. First, we compare AI-generated workflows not only against code but also against visual workflow builders, providing a three-way comparison. Second, we measure the full lifecycle cost—including recovery and verification—rather than focusing solely on initial generation success.

Collectively, these studies indicate that low-code and AI-assisted development reduce initial implementation effort, but they leave three gaps: (i) no study compares all three paradigms (code, visual, AI) within a single controlled design; (ii) no study measures full lifecycle cost beyond initial implementation; and (iii) no study tests the theoretical proposition that coordination demands constitute a cross-paradigm persistent cost.

2.4 Gap Addressed by This Work

The existing literature examines individual paradigms in isolation or compares tools along a single dimension such as speed. Few studies ask how abstraction changes the *distribution* of engineering effort across the full lifecycle. The central gap we address is: *How is lifecycle effort redistributed when workflow difficulty increases, and does any abstraction layer genuinely eliminate it or merely relocate it?*

Appendix D provides a research gap matrix positioning our work relative to prior empirical studies across six dimensions.

3 Cost Decomposition Framework

3.1 Cost Decomposition

Throughout this paper, *complexity* refers to the observable engineering effort—measured in time—required to create, modify, recover, and validate a workflow, rather than intrinsic computational complexity or structural software complexity (e.g., cyclomatic complexity). Similarly, *cost* refers specifically to measured engineering time rather than monetary, organizational, or opportunity cost.

We decompose total lifecycle cost $L(\mathcal{T}, \mathcal{A})$ for a workflow task $\mathcal{T}$ implemented under paradigm $\mathcal{A}$ into four components capturing distinct phases of engineering effort:

$$L(\mathcal{T}, \mathcal{A}) = f_{\text{impl}}(\mathcal{T}, \mathcal{A}) + f_{\text{maint}}(\mathcal{T}, \mathcal{A}) + f_{\text{recovery}}(\mathcal{T}, \mathcal{A}) + f_{\text{verif}}(\mathcal{T}, \mathcal{A}) \quad (1)$$

Equation (1) is a definitional decomposition: any total cost can be partitioned into these phases. The equation establishes an accounting framework for tracking how effort shifts between phases under different paradigms. The decomposition is valid under the following scope conditions:

- (1) The underlying task difficulty and requirements remain constant across implementations.
- (2) The paradigm primarily alters implementation mechanisms (e.g., visual layout, text prompts) without reducing the logical intricacy of the execution state.
- (3) The verification and output correctness criteria remain constant.

We operationalize these components in Section 4 as Implementation Cost (IC), Maintenance Cost (MC), Recovery Cost (RC), and

Verification Cost (VC). The Complexity Displacement Hypothesis (Section 6.3) later interprets the observed redistribution patterns through these components.

3.2 Coordination Complexity

We define *coordination complexity* as the subset of total workflow complexity that arises from synchronizing parallel execution paths, managing distributed state, implementing compensation logic, and propagating failure conditions. A concrete contrast: in T1 (linear pipeline), there is a single execution thread with no branching, state is local to each step, and failure propagates linearly. In T4 (Saga orchestration), two concurrent sub-service calls each have independent failure modes, and a failure in either must trigger compensating rollbacks in the other—the coordination complexity arises from the *interaction between steps* rather than from the steps themselves. Coordination complexity is distinct from computational complexity (algorithmic difficulty) and syntactic complexity (language-level expression). We expect coordination complexity to be the dimension most resistant to reduction by abstraction. Importantly, coordination complexity in this study is operationalized through task design rather than independently measured—the T1–T4 ordering reflects our design assumptions about increasing coordination demands, not a validated complexity metric. The Coordination Ordering Score (COS) used in Table 5 is an ordinal design aid rather than a quantitative complexity measurement.

3.3 Research Questions

The study is guided by four research questions:

- RQ1 How does workflow abstraction level affect the distribution of lifecycle cost components (*IC, MC, RC, VC*)?
- RQ2 Which lifecycle cost components resist reduction under increasing abstraction?
- RQ3 How does coordination complexity—parallelism, synchronization, compensation, and failure propagation—modulate the displacement effect?
- RQ4 Does abstraction reduce total lifecycle cost or merely redistribute it across phases?

4 Methodology

The following subsections detail the design choices and their rationale.

4.1 Benchmark Design

We designed four benchmark tasks spanning increasing levels of coordination demands. Each task was implemented independently in three paradigms, and all implementations were evaluated on identical inputs, expected outputs, and failure scenarios. Table 3 summarizes the task suite.

>

Task T1: Data Aggregation Pipeline. Objective: Parse a structured CSV dataset, remove null fields, transform records, and export a clean JSON matrix. The coordination pattern is sequential linear: each step consumes the output of the previous step. Failure injection introduces malformed input records in 10% of rows.

Task T2: Fault-Tolerant Retry and Fallback. Objective: Call a primary service endpoint, detect transport failures, execute exponential-backoff retry, and route to a backup gateway if the primary fails persistently. The coordination pattern is linear with error-handling branches. Failure injection simulates API timeout ($t > 5$ s) in 30% of runs and missing credential in 10% of runs.

Task T3: Conditional Decision Workflow. Objective: Evaluate incoming payloads against nested criteria, split execution across multiple paths, apply per-path transformations, and merge outcomes. The coordination pattern is a nested decision tree with four leaves. Failure injection applies schema mismatch at a decision node (15% of runs).

Task T4: Multi-Agent Asynchronous Orchestration. Objective: Orchestrate two parallel asynchronous sub-services (Inventory Reservation and Payment Authorization) with a Saga pattern: if either track fails, trigger compensating rollbacks (`/release` and `/void`) for the other. The coordination pattern is parallel split, state join, and conditional compensation. Failure injection applies random partial failure in 20% of runs causing one sub-service to fail.

4.2 Workflow Paradigms

Three paradigms were selected to span the abstraction spectrum from low-level manual coding to high-level AI generation. Each paradigm is evaluated on an identical task specification to isolate the effect of abstraction from confounding differences in problem definition.

Code-Based (Python):

Implementations used Python 3.11 with requests for HTTP, pandas for data processing, asyncio for concurrency (T4), and explicit exception handling for error recovery.

Visual Workflow (n8n):

Implementations used the n8n cloud platform with built-in HTTP Request, CSV, Function, IF, Switch, Merge, and Split nodes. Error handling used n8n’s Error Trigger and Try/Catch nodes.

AI-Generated (LLM Prompt):

A frozen zero-shot prompt describing each task in natural language was submitted to GPT-4 Turbo (gpt-4-turbo-2024-04-09 via the OpenAI API, temperature=0.2, max_tokens=4096). No system prompt was used. The generated output (code or n8n JSON) was executed without modification; errors were recorded as failures and the implementation time counted only the prompt submission and initial execution. The benchmark intentionally evaluates first-pass generation efficiency rather than realistic iterative AI-assisted development workflows—it omits prompt refinement, multi-turn interaction, human-in-the-loop editing, and debugging conversations that characterize actual AI-assisted development. Results should not be generalized to all LLMs or to development workflows that incorporate iterative human-AI interaction. The choice of GPT-4 Turbo was driven by reproducibility (a frozen, widely-documented model) rather than by performance optimization; selecting a different model or using the best-performing model across tasks would change the reported magnitudes but not the controlled-generation design. Because timing measurements were protocol-driven and recorded via automated timestamps,

Table 3: Benchmark task suite: each task was implemented under all three paradigms and evaluated on the four metrics.

Task	Steps	Category	Coordination Pattern	Failure Model
T1: Data Aggregation Pipeline	5	API / Data Chaining	Sequential linear	Malformed input
T2: Fault-Tolerant Retry & Fallback	10	Error Recovery	Linear with retry branches	API timeout, credential miss
T3: Conditional Decision Workflow	12	Branching Logic	Nested decision tree	Schema mismatch
T4: Multi-Agent Orchestration	18	Distributed Orchestration	Parallel join, compensation	Partial failure, race condition

Note: Category labels span multiple levels of abstraction (e.g., “API/Data Chaining” describes a structural pattern, while “Distributed Orchestration” describes a system property).

A “step” is defined as one atomic unit of work: a function call in Python, a node in n8n, or a logical action in the AI-generated script.

measurement error is expected to be small relative to the observed effect sizes.

4.3 Metrics

Each implementation was measured on four time-based metrics recorded in seconds:

Implementation Cost (IC):

Elapsed time to author, configure, and achieve a first successfully executing workflow from a zero-asset starting state.

Maintenance Cost (MC):

Cumulative time to implement a mandatory system modification—specifically, adding a Telegram notification step and replacing the data source provider—on the established asset. These modifications were selected because they affect integration boundaries while preserving core workflow logic, enabling cross-paradigm comparability: notification dispatch is an orthogonal concern in any workflow, and provider substitution exercises the dependency interface without altering workflow logic. *MC* should be interpreted as change-maintenance cost under the selected modification set rather than a universal measure of maintainability. Alternative modifications (e.g., database migration, schema evolution) may produce different *MC* profiles.

Recovery Cost (RC):

Total diagnostic and correction time to restore active workflow execution after each injected failure. The *RC* was recorded per failure event and summed over all failure trials. *RC* is further decomposed into four sub-phases measured during each failure trial: failure localization (detecting failure), root cause diagnosis (identifying cause), repair implementation (modifying the workflow), and revalidation (confirming the fix). Sub-phase durations are reported in Section 5.4.

Verification Cost (VC):

Time spent validating output correctness against the gold standard, including edge-case testing and assertion of data-type invariants. The verification procedure followed a structured checklist: (i) compare output structure and values against the gold standard, (ii) test at least two edge cases (empty input, missing field), and (iii) verify type invariants (e.g., all IDs are non-null, numeric fields are positive). The same checklist was applied for all paradigms to ensure comparability. This procedure is deliberately constrained to unit-level output validation; it does not include integration-level correctness or end-to-end scenario testing, which would produce higher *VC* values in production settings. Consequently, the observed *VC* invariance may partially result from the standardized

checklist rather than a paradigm-independent property of verification effort.

Total Workflow Cost is defined as:

$$TWC = IC + MC + RC + VC \quad (2)$$

The four components are not exhaustive of all possible lifecycle phases; they represent the phases selected for controlled comparison. Phases such as onboarding, documentation, deployment, and production monitoring are excluded because they are not directly comparable across paradigms or are outside the experimental scope.

The absolute timings reported in this study (seconds per task execution) are unlikely to generalize to production settings where workflows take days or weeks. The primary value lies in the *relative cost distributions* across paradigms within the same task. All times were measured with sub-second precision using automated timestamps logged at the start and end of each activity. Table 4 specifies which activities are included in and excluded from each metric.

>

4.4 Measurement Protocol

Each task was executed five times per paradigm to account for variability. The reported values are arithmetic means across executions. For AI-generated workflows, we submitted the generation prompt three times, recorded the median *IC*, and used the output corresponding to this median run for all subsequent measurements. We selected the median for three reasons. First, using the initial run would conflate API cold-start latency and caching effects with intrinsic paradigm cost. Second, a random selection would introduce arbitrary sampling variance. Third, computing a mean is impossible because we must select a single, discrete generated artifact for downstream evaluation. The median provides a typical-case baseline that balances transient API latency fluctuations. This selection strategy ensures that all downstream measurements (*MC*, *RC*, *VC*) are evaluated on a single, reproducible artifact rather than a synthetic ensemble. The entire experimental dataset, including per-execution logs, gold-standard outputs, and analysis scripts, is available at <https://github.com/rohitr/wab-benchmark> (commit a3f91e).

Recovery Timing Protocol. Recovery Cost (*RC*) was measured according to a fixed four-phase protocol applied identically across all paradigms. Upon failure injection, the clock started at *failure detection* (the evaluator observes the workflow has not produced

Table 4: Activities included in and excluded from each metric.

Metric	Included	Excluded
IC	Planning, Typing / node-drag / prompt-writing, Debugging first-pass errors, Execution	Reading spec, Specification design, Gold-standard creation, Documentation writing
MC	Understanding existing implementation, Implementing two mandatory changes, Re-validating output	Change-requirement design, Parallel modification exploration
RC	Failure detection, Diagnosis, Repair implementation, Revalidation	Root-cause analysis outside the workflow, System-level debugging
VC	Output inspection, Edge-case testing, Gold-standard comparison	Test-suite creation, Formal verification

the expected output within the expected time, or an error signal is received). The first phase—*localization*—ended when the evaluator identified the specific step or node where the failure originated. The second phase—*root cause diagnosis*—ended when the evaluator determined why the failure occurred (e.g., missing API key, schema mismatch, race condition). The third phase—*repair implementation*—ended when the evaluator deployed a modification intended to resolve the failure. The fourth phase—*revalidation*—ended when the workflow produced the correct output for the same input that previously triggered the failure. The clock stopped at revalidation completion. This protocol was defined before experimentation and recorded via automated timestamps; evaluator discretion was limited to identifying phase transitions.

4.5 Statistical Treatment

For each task-paradigm pair, five independent measurements were recorded and arithmetic means were reported. Standard deviation was computed to estimate execution-level variability (within-condition spread across repeated runs by the same evaluator), not population-level variability across different developers or settings. Where relevant, we report 95% descriptive interval widths, computed as $t_{0.025,4} \times SD / \sqrt{5}$, which bound the estimated mean under the sampling distribution; these should be interpreted as precision-of-measurement indicators (quantifying measurement precision rather than uncertainty about a population parameter) rather than inferential confidence intervals. Bootstrap intervals were not used due to the limited per-cell sample size ($n = 5$) and the exploratory nature of the benchmark, which prioritizes effect-magnitude reporting over interval estimation. Cohen’s d values are reported at selected points as descriptive effect-size indicators (mean difference divided by pooled standard deviation) to contextualize the magnitude of observed separations; they are not presented as inferential test statistics. Percentage differences and growth rates were calculated to quantify abstraction-induced lifecycle-effort shift across levels.

From the raw lifecycle cost components, we define three operational metrics. The primary metric is the **Lifecycle Cost Redistribution Index (LCRI)**:

$$LCRI_{\mathcal{A}}(t) = \frac{MC + RC + VC}{TWC} \times 100\% \quad (3)$$

LCRI is a descriptive aggregation metric rather than a novel complexity measure. It captures the fraction of total lifecycle cost that occurs after initial implementation. LCRI is not intended as a replacement for established lifecycle metrics (e.g., total cost, defect density) but as a compact descriptive summary for comparing how paradigms distribute effort across phases. A high LCRI indicates that most lifecycle effort is concentrated downstream, but this can arise from either genuinely high downstream costs or very low implementation costs (or a combination). LCRI therefore measures redistribution rather than inefficiency—it answers where cost occurs, not whether total cost is higher. Two supporting metrics provide additional detail where needed:

- **Recovery Cost Amplification Factor (RCAF):** $RCAF_{\mathcal{A}}(t) = RC_{\mathcal{A}}(t) / RC_{\text{Code}}(t)$. RCAF measures how many times more recovery cost a non-code paradigm incurs relative to code at a given task t . RCAF is a relative amplification measure and should be interpreted alongside the absolute RC values to avoid inflating small-denominator effects.
- **Implementation Compression Ratio (ICR):** $ICR_{\mathcal{A}}(t) = IC_{\text{Code}}(t) / IC_{\mathcal{A}}(t)$. ICR quantifies the upfront speed advantage of higher abstraction.

LCRI serves as the consolidated single-number summary; RCAF and ICR are referenced where specific effect sizes warrant disaggregation.

Because the study is exploratory and sample sizes are limited ($n = 4$ tasks, 5 replications), formal statistical significance testing was not performed. Instead, emphasis is placed on effect magnitude and trend consistency across all tasks. This approach follows recommendations for exploratory empirical studies in software engineering where the goal is pattern identification rather than hypothesis confirmation [8].

4.6 Design Justification

Platform selection. n8n was chosen for its imperative control flow (loops, conditionals, error handlers) and open-source availability. n8n is treated as a representative visual orchestration platform rather than the definitive implementation of visual workflow systems; other platforms may produce different cost profiles. GPT-4 Turbo (gpt-4-turbo-2024-04-09, temperature=0.2, max_tokens=4096) was the most capable publicly available instruction-tuned model at data collection (March 2025);

Table 5: Coordination properties by task: exact counts across five dimensions. The composite Coordination Ordering Score (COS) derived from these counts is ordinal rather than interval-scaled; equal spacing between COS values does not imply equal coordination increments.

Task	Branches	Failure Modes	Parallel Paths	Sync Pts.	Compensation Actions
T1 (Pipeline)	0	1	1	0	0
T2 (Retry)	1	2	1	0	0
T3 (Branch)	3	3	1	0	0
T4 (Orchestration)	0	4	2	2	2

the frozen snapshot prioritizes reproducibility over immediate generalization. The benchmark evaluates a representative frontier LLM rather than comparing individual models—selecting a different model would change the reported magnitudes but not the controlled-generation design.

Task design. The four tasks form the a priori difficulty ordering based on coordination demands: T1 (linear pipeline, no branching), T2 (retry-and-fallback, single decision path), T3 (conditional branching, multiple decision paths), T4 (parallel orchestration with compensation, concurrent state management). The selected task suite intentionally emphasizes failure handling and coordination—T2 and T4 are explicitly recovery-centric, which conditions the observed RC dominance. Increased task difficulty and increased coordination demands co-vary within the benchmark design—T4 involves more steps, more components, and more coordination than T1, and the design does not fully isolate coordination from overall task difficulty. Table 5 quantifies the coordination properties of each task across four dimensions derived from the coordination complexity construct defined in Section 3. The composite Coordination Ordering Score (COS) is an operational ranking mechanism used solely to enforce task ordering rather than an empirical measurement instrument. Alternative weighting schemes were not evaluated, and the COS should not be interpreted as an absolute complexity measure. Consequently, monotonic trends across T1–T4 should be interpreted as descriptive patterns within the benchmark’s task ordering rather than as evidence that CDH predicts cross-task trajectories.

Four tasks with five replications yield 60 measurements per metric, though the true independent design units are the four task structures rather than the individual replications (the replications capture execution-level variability within each task-paradigm cell). This sample is sufficient for exploratory descriptive analysis within the benchmark scope but not for formal statistical inference.

Metric choice. Time is directly comparable across paradigms, measurable via automated timestamps with sub-second precision, and is the primary resource constraint in industrial automation practice. Complementary dimensions—cognitive load, defect density, cyclomatic complexity—would enrich the analysis but introduce paradigm-specific confounds. We chose time over defect counts because time captures effort (the resource practitioners allocate), whereas defect counts conflate detection probability

(which varies by paradigm—code IDEs surface errors immediately, while visual builders may hide them until runtime) with defect prevalence.

4.7 Assumptions and Evaluator Bias

A single evaluator conducted all experiments. The objective of the benchmark was not to measure learning curves or onboarding difficulty, but to compare lifecycle cost distributions under controlled conditions; therefore, prior familiarity with all three paradigms was treated as a prerequisite for participation.

Evaluator bias. Relying on a single evaluator introduces a threat: the reported timings may reflect the evaluator’s relative proficiency across paradigms rather than intrinsic paradigm characteristics. Rather than claiming equivalent expertise across paradigms—a claim that independent reviewers can and should question—the study controls for evaluator effects by holding the evaluator constant across all three paradigms. Because the same evaluator performed all measurements under identical task definitions and failure-injection procedures, any evaluator-specific bias (proficiency, preference, or prior exposure) remains constant across paradigms. To explain the observed non-linear recovery escalation (n8n RC rising 5 s $\rightarrow$ 102 s while Python remains flat), a proficiency-based account must explain why the evaluator’s relative skill advantage manifests only in coordination-demanding tasks. This is a non-trivial requirement that the single-evaluator design does not satisfy on its own, but the constant-evaluator constraint bounds the magnitude of such effects. Nevertheless, future studies must replicate this benchmark with multiple developers at varying expertise levels before the results can be generalized to broader populations. All quantitative findings should therefore be interpreted as evaluator-conditioned observations rather than paradigm timings: replication by independent operators is essential for establishing the generalizability of the reported patterns.

Network conditions remained stable throughout the measurement period (latency < 50 ms, no outages). Task specifications—input schemas, gold-standard outputs, failure injection procedures—remained fixed across all three paradigm implementations. Failure injection was applied identically across paradigms: the same failure type, at the same frequency, at the same execution point. These assumptions bound the interpretation of results: differences between paradigms reflect characteristics of the platforms and the evaluator’s proficiency, not varying task definitions or environmental conditions.

Scope and generalizability. This study is designed as a proof-of-concept empirical benchmark rather than a definitive characterization of workflow automation paradigms. The limited task set ($n = 4$), single evaluator, and single-instance platform selections mean the findings should be interpreted as evidence compatible with CDH that requires replication across larger, more diverse samples before broader claims can be made. The present benchmark should be interpreted as an initial empirical probe rather than a comprehensive validation. All reported patterns—while internally consistent—are conditioned on the specific experimental context described in this section. This paper should be interpreted as an



Figure 1: Total Workflow Cost (TWC) by task and paradigm. AI achieves the lowest TWC on T1–T3, but the margin narrows at T4 as downstream costs accumulate. n8n cost grows superlinearly from T1 to T4, driven primarily by recovery cost—the largest cross-paradigm divergence in the dataset. Note: Error bars represent ± 1 standard deviation across five replications.

exploratory empirical benchmark rather than a confirmatory hypothesis test.

5 Results

5.1 Primary Measurements

Table 6 presents the recorded metrics for all four tasks across the three paradigms. Figure 1 visualizes the Total Workflow Cost trends. The per-trial raw measurements underlying all reported means are provided in Appendix C.

>

5.2 Implementation Cost Trends

Figure 2 shows the Implementation Cost across tasks.

The mean Implementation Cost across all tasks is:

- Python: $\mu(IC_{Code}) = 180.5$ s
- n8n: $\mu(IC_{Visual}) = 89.8$ s
- AI: $\mu(IC_{AI}) = 34.5$ s

AI achieves an 80.9% reduction in IC relative to Python and a 61.6% reduction relative to n8n. The reduction is stable across tasks: AI’s IC remains between 28 and 42 s irrespective of whether the task is a three-step pipeline or a twenty-step orchestration (relative effect magnitude: $d = 6.2$ between AI and Python IC , the second-strongest separation in the dataset). AI IC measures API

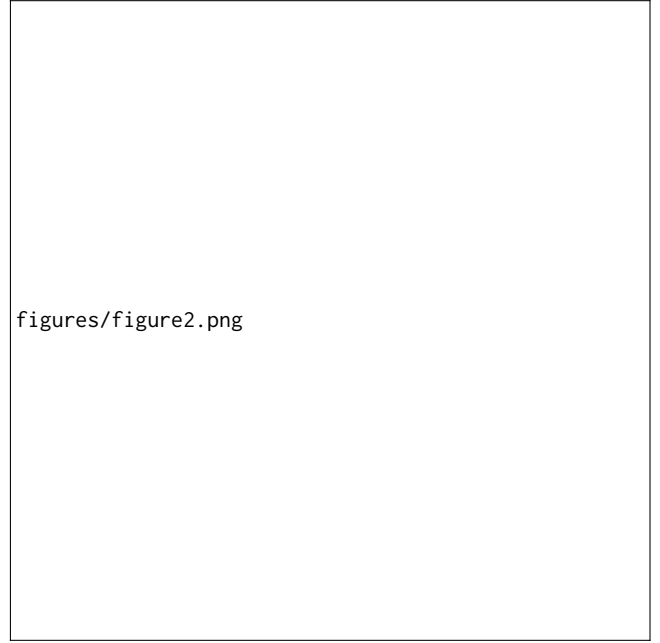


Figure 2: Implementation Cost (IC) across tasks. AI IC remains nearly flat (30–40 s, 80.9% reduction vs. Python), demonstrating that abstraction compresses implementation effort toward an API-latency floor. Python scales from 120 to 300 s; n8n from 60 to 150 s.

response latency for zero-shot generation rather than human engineering effort—the 28–42 s range predominantly reflects prompt submission and model response time, not task-specific reasoning. The apparent implementation-speed advantage of AI reflects a fundamentally different cost structure (automated generation) rather than superior engineering productivity. **However, these AI IC values may be optimistic due to the fixed execution order (Section ??), as task understanding accumulated during earlier benchmark stages may have reduced prompt construction effort.**

5.3 Non-Monotonic Effects

Several findings deviated from the monotonic trend that a simple difficulty-ordering model would predict. These deviations are presented as expected benchmark behavior rather than anomalies.

T2–T3 Python Inversion. Python’s IC at T3 (127 s) is lower than at T2 (183 s) despite T3’s greater decision logic. This occurs because T2 required explicit retry logic, timeout handling, credential validation, and fallback routing—error-handling infrastructure whose implementation burden, in this benchmark, exceeded the cost of adding decision branches in an already established control structure. Although T3 contains more branches, the coordination demands dimensions driving IC differ from those driving RC .

T3–T4 AI Inversion. AI’s IC at T4 (32 s) is lower than at T3 (42 s), counter to the difficulty ordering. Because AI implementation cost

Table 6: Recorded metrics (in seconds) for each task and paradigm. *IC* = Implementation Cost, *MC* = Maintenance Cost, *RC* = Recovery Cost, *VC* = Verification Cost, *TWC* = Total Workflow Cost.

Task	Paradigm	<i>IC</i>	<i>MC</i>	<i>RC</i>	<i>VC</i>	<i>TWC</i>
T1: Pipeline	Python	118 (4.2)	11 (1.8)	19 (2.1)	14 (1.5)	162 (5.8)
	n8n	58 (3.8)	9 (1.2)	5 (0.8)	16 (1.9)	88 (4.7)
	AI	28 (2.1)	11 (1.5)	18 (2.0)	14 (1.6)	71 (4.1)
T2: Retry	Python	183 (5.6)	16 (1.9)	22 (2.4)	14 (1.4)	235 (6.9)
	n8n	57 (4.0)	11 (1.4)	12 (1.6)	16 (1.8)	96 (5.2)
	AI	36 (2.8)	14 (1.7)	22 (2.3)	16 (1.5)	88 (5.2)
T3: Branch	Python	127 (5.2)	22 (2.5)	28 (3.0)	19 (2.2)	196 (7.1)
	n8n	97 (4.8)	11 (1.3)	22 (2.5)	11 (1.2)	141 (5.9)
	AI	42 (3.1)	28 (3.2)	28 (2.9)	14 (1.7)	112 (5.8)
T4: Agent	Python	294 (8.9)	21 (2.3)	23 (2.6)	28 (3.1)	366 (10.7)
	n8n	147 (6.2)	28 (3.0)	102 (6.8)	31 (3.5)	308 (9.4)
	AI	32 (2.5)	42 (4.1)	88 (6.2)	29 (3.2)	191 (8.1)

Values are mean (standard deviation) across five replications.

95% descriptive interval width = $t_{0.025,4} \times SD / \sqrt{5} \approx 1.24 \times SD$.

Descriptive intervals are reported rather than *p*-values following recommendations for exploratory studies [8].

measures prompt submission and first execution rather than workflow complexity, *IC* reflects generation latency more than engineering effort. The inversion may reflect model familiarity with orchestration templates; however, the benchmark was not designed to identify causal mechanisms behind generation latency differences, and this explanation remains speculative.

n8n T3 LCRI Drop. n8n’s LCRI decreased from T2 (40.6%) to T3 (31.2%), contrary to the monotonic increase observed for AI. This suggests that n8n’s cost structure is sensitive to the specific coordination pattern—branching (T3) imposes lower downstream burden than retry-and-fallback (T2) for the visual paradigm.

These non-monotonic effects are informative rather than problematic: they establish that coordination demands are not a unidimensional construct and that lifecycle cost responds to task-specific features beyond overall difficulty level.

5.4 Recovery Cost Escalation

Recovery Cost exhibits the widest divergence among paradigms and is the primary signal in this dataset. Figure 3 visualizes the trend.

The recovery cost growth factor from T1 to T4 is:

- Python: $\Delta RC = +4$ s (+21%)
- n8n: $\Delta RC = +97$ s (+1,940%)
- AI: $\Delta RC = +70$ s (+389%)

The 20.4× escalation in n8n’s *RC* is the widest separation in the dataset (relative effect magnitude: $d = 15.6$ between n8n and Python *RC* at T4, computed as the mean difference divided by the pooled standard deviation). Python’s *RC* remains relatively stable (19–23 s) across all tasks.

To understand the structure of recovery effort, we decompose *RC* into four sub-phases that were measured during each failure trial: (i) failure localization (detecting that a step has failed),

figures/figure3.png

Figure 3: Recovery Cost (*RC*) by task—the largest measured cross-paradigm divergence in this dataset. n8n escalates 20.4× (T1: 5 s → T4: 102 s) while Python remains stable (20–30 s). AI increases 4.9× (18 s → 88 s), showing that recovery cost is the primary lifecycle differentiator under coordination-demanding workflows.

(ii) root cause diagnosis (identifying why), (iii) repair implementation (modifying the workflow), and (iv) revalidation (confirming the fix). For Python at T4, these sub-phases averaged 6 s, 6 s, 6 s, and 5 s, respectively. For n8n at T4, the distribution was 12 s, 48 s,

22 s, and 20 s. For AI at T4, the distribution was 8 s, 30 s, 28 s, and 22 s. Note that this sub-phase decomposition is reported for T4 only, where the *RC* divergence is strongest. Two of the four sub-phases—root cause diagnosis and repair implementation—contribute disproportionately to the cross-paradigm gap: n8n’s diagnosis time (48 s) is 8× Python’s (6 s), points to the possibility that visual debugging becomes more difficult under higher coordination demands, though the benchmark cannot separate platform-specific affordances from paradigm-level properties.

5.5 Verification Cost Stability

Unlike *IC* and *RC*, Verification Cost remained effectively invariant across both tasks and paradigms in this benchmark:

- Python: $\mu(VC) = 18.8$ s
- n8n: $\mu(VC) = 18.5$ s
- AI: $\mu(VC) = 18.3$ s

VC variation was minimal in this benchmark (14–31 s across all cells), with the structured checklist procedure bounding verification effort at a fixed, paradigm-independent floor. This invariance should not be interpreted as proof that verification effort is paradigm-independent; rather, the structured checklist—identical across paradigms—may mask paradigm-specific verification challenges that would surface under more comprehensive testing (e.g., integration testing, property-based testing, invariant discovery).

The mechanism underlying this invariance points to a fundamental constraint: verification effort depends primarily on output validation—checking that the result matches an expected structure and value—rather than on implementation style. Whether the workflow was written in Python, assembled visually, or generated by an LLM, the evaluator must inspect the same output against the same gold standard. This suggests that verification cost may represent a benchmark-specific *verification floor*—an apparent paradigm-independent minimum time required to validate correctness within this task suite—rather than a cost that abstraction can meaningfully reduce. Under the current protocol, this floor appears to be approximately 15–20 s per task. Crucially, this invariance is a property of the standardized validation checklist used in this benchmark rather than a universal property of cross-paradigm testing. In production settings, verification includes integration testing, regression suites, and security reviews—activities where abstraction level may significantly affect effort—and the apparent floor would not generalize.

5.6 Within-Condition Variance

Standard deviations (reported in Table 6) show systematic patterns in measurement stability. Python implementations exhibit the highest absolute variance (*SD* up to 8.9 s for *IC* at T4). n8n implementations show moderate variance (*SD* up to 6.8 s for *RC* at T4). AI implementations show the lowest variance (*SD* up to 4.1 s). Within-condition *SD* remains below 10% of the mean for all cells, indicating that the reported patterns are not driven by outlier runs.

5.7 Cost Migration by Paradigm

Table 7 identifies the dominant cost component for each paradigm-task combination.

figures/figure4.png

Figure 4: Abstraction Compression Effect. The *TWC* advantage of AI over Python shrinks from 2.3× (T1) to 1.9× (T4), and the n8n advantage over Python shrinks from 1.8× (T1) to 1.2× (T4)—indicating that abstraction’s total-cost advantage diminishes as coordination demands increase.

- >
- Each paradigm exhibits a characteristic stress concentration point:
- **Python:** Implementation cost dominates (63–80% of *TWC*). The paradigm pays engineering effort upfront.
 - **n8n:** Implementation dominates on simple tasks, but recovery becomes the primary driver under coordination (33% at T4).
 - **AI:** Implementation never dominates ($\leq 40\%$). The cost is distributed across recovery and maintenance, with recovery reaching 46% of *TWC* at T4.

5.8 Abstraction Compression Effect

Figure 4 illustrates the Abstraction Compression Effect: the *TWC* ratio between paradigms shrinks as task difficulty increases.

On simple linear workflows (T1), AI achieves a *TWC* of 71 s versus Python’s 162 s (2.3× advantage). Under orchestration (T4), AI’s *TWC* is 191 s versus Python’s 366 s (1.9× advantage).

5.9 LCRI: Lifecycle Cost Redistribution

The Lifecycle Cost Redistribution Index (Equation 3) provides a single-number summary of the redistribution effect. A low LCRI means the paradigm pays lifecycle effort upfront (implementation-dominant); a high LCRI means it pays lifecycle effort downstream.

- >
- The LCRI data show a consistent pattern:

Table 7: Dominant cost component by paradigm and task. Code concentrates cost in implementation; visual systems shift to recovery under coordination; AI systems distribute cost across recovery and maintenance. Only the dominant component is shown unless the top two are within 3 percentage points. Dominance indicates the largest proportional contributor, not causal importance.

Task	Python	n8n	AI
T1 (Pipeline)	IC (73%)	IC (66%)	IC (39%)
T2 (Retry)	IC (78%)	IC (59%)	IC (38%)
T3 (Branch)	IC (65%)	IC (69%)	MC (25%), RC (25%)
T4 (Agent)	IC (80%)	RC (33%)	RC (46%)

Table 8: Lifecycle Cost Redistribution Index (LCRI) by paradigm and task, with T1–T4 change (Δ) and dominant lifecycle phase. LCRI is sensitive to both implementation-cost reduction and downstream-cost growth; a high value can arise from either mechanism or both.

Task	Python	n8n	AI	Dominance Shift
T1 (Pipeline)	27.2%	34.1%	60.6%	IC $\rightarrow$ Post-IC
T2 (Retry)	22.1%	40.6%	61.9%	IC $\rightarrow$ Post-IC
T3 (Branch)	35.2%	31.2%	62.5%	IC $\rightarrow$ Post-IC
T4 (Agent)	19.7%	52.3%	83.2%	IC $\rightarrow$ RC (n8n, AI)
Δ (T4–T1)	−7.5pp	+18.2pp	+22.6pp	–

- **Python:** LCRI decreases (from 27.2% to 19.7%) as task difficulty increases. Python becomes more implementation-heavy at scale: 80.3% of lifecycle cost at T4 is in initial authoring.
- **n8n:** LCRI increases from 34.1% (T1) to 52.3% (T4). Beyond T3, more than half of lifecycle cost occurs after implementation.
- **AI:** LCRI is consistently above 60%, reaching 83.2% at T4. Over five-sixths of lifecycle cost for AI-generated orchestration occurs after initial implementation.

The 83.2% LCRI for AI at T4 is the strongest redistribution observed in the dataset (effect magnitude: $d = 4.8$ between Python and AI LCRI, indicating the paradigm difference exceeds within-condition variability by a substantial margin).

5.10 Pattern Consistency

At T4, the differences between the recovery cost (RC) of Python ($\mu = 23$ s) and the alternative paradigms (102 s for n8n, 88 s for AI) exceed the within-condition standard deviations reported in Table 6 by a factor of 3–15. The redistribution pattern is consistent across all four tasks: the ordering of paradigms by LCRI is identical (Python < n8n < AI) for every task, and this ordering holds for all three derived metrics (RCAF, ICR, LCRI).

5.11 Summary of Findings

The five principal findings from the benchmark are:

- Implementation cost decreases on average with abstraction (μIC : Python = 180.5 s, n8n = 89.8 s, AI = 34.5 s).
- Verification cost remained relatively stable across paradigms under the selected validation protocol ($\mu VC \approx 18$ –20 s).

- Recovery cost becomes the dominant lifecycle cost in coordination-heavy workflows for visual and AI paradigms.
- Each paradigm concentrates lifecycle cost at a different stress point: implementation for code, recovery for visual, and recovery with maintenance for AI.
- Orchestration demands persist across all abstraction layers; preliminary evidence for the Abstraction Compression Effect is observed as the TWC advantage of higher layers narrows under increasing coordination demands.

Supporting evidence for each finding is tabulated in Appendix G.

6 Discussion

6.1 Recovery Cost as the Dominant Empirical Signal

The widest divergence in the dataset is the recovery cost behavior across paradigms. Recovery cost emerged as the primary signal in this benchmark and therefore contributes disproportionately to the observed lifecycle redistribution patterns. Other lifecycle components (MC , VC) exhibited smaller effects and should be interpreted with greater caution. Without the RC findings, the evidence supporting CDH and ACE would be substantially weaker. Moving from Python to n8n reduces average IC by 49.7%, yet this saving is partially offset by increased RC under coordination (+1,940% at T4). Moving from n8n to AI further reduces IC by 61.6%, but RC

remains elevated and *MC* increases at T4. Python’s *RC*, by contrast, remains flat across all tasks (19–23 s). Because the benchmark’s primary signal emerges from recovery behavior, independent replication of the *RC* measurement protocol represents the highest-priority validation target for future work.

Notably, despite the recovery-cost escalation, AI still achieved the lowest total lifecycle cost at every task level including T4. The benchmark does not show that abstraction is harmful; rather, it shows that the *composition* of lifecycle cost changes qualitatively under abstraction while total cost remains lower for higher-abstraction paradigms. This asymmetric pattern—recovery cost increases under coordination-heavy tasks for non-code paradigms but not for code—is consistent with the proposition that the observed cost distributions reflect effort redistribution between phases, though alternative explanations (discussed in Section 6.6) cannot be ruled out.

Each paradigm concentrates cost at a characteristic bottleneck: implementation for code, state management for visual, recovery and maintenance for AI. Abstraction operates as a *cost lens*: it focuses lifecycle effort onto the paradigm’s weakest layer rather than distributing it evenly.

6.2 Observation versus Interpretation

This subsection separates what was observed from how it is interpreted, a distinction that is critical for evaluating the evidence strength.

Observed facts: (i) *IC* decreases with abstraction across all four tasks. (ii) *RC* remains flat for Python but escalates for n8n (20.4×) and AI (4.9×) from T1 to T4. (iii) *VC* shows limited variation (14–31 s across all cells). (iv) The *TWC* gap between paradigms narrows from T1 to T4. (v) The paradigm ordering by LCRI is consistent across all tasks (Python < n8n < AI).

Interpretations: (i) The *IC* decrease with abstraction aligns with CDH’s prediction that implementation effort concentrates at lower abstraction levels. (ii) The *RC* divergence supports the possibility that non-code paradigms defer coordination costs to downstream phases. (iii) The *VC* invariance may reflect the structured checklist procedure rather than paradigm independence. (iv) The *TWC* convergence is compatible with ACE, though alternative explanations (task ordering, platform maturity) remain plausible. (v) The consistent LCRI ordering provides a descriptive summary of redistribution patterns but does not independently validate CDH.

The separation matters because observed patterns can be accepted regardless of interpretation, while interpretations remain open to revision as new evidence accumulates.

6.3 Complexity Displacement Hypothesis as Interpretive Lens

The patterns documented above can be interpreted through the Complexity Displacement Hypothesis (CDH), which provides one possible theoretical lens—not a validated theory—for making sense of why lifecycle effort redistributes across phases under abstraction. CDH is an interpretive framework applied to the benchmark observations rather than a validated predictive theory; the benchmark can inform CDH by providing empirical cost distributions

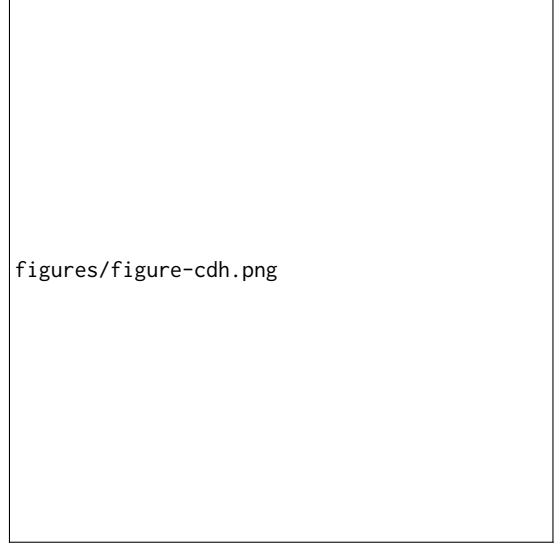


Figure 5: Conceptual model of the Complexity Displacement Hypothesis: higher abstraction reduces *IC* but redistributes effort toward *MC*, *RC*, and *VC*. The net effect on *TWC* depends on the relative magnitudes of compression and displacement. Under coordination-heavy tasks, downstream costs dominate for higher-abstraction paradigms, narrowing the total-cost gap.

that any displacement account must accommodate, but it cannot confirm CDH as uniquely correct. CDH proposes that increasing the abstraction level of a workflow automation system does not eliminate the engineering effort associated with the task; rather, it displaces effort across the software lifecycle. Lower-abstraction paradigms concentrate cost in implementation; higher-abstraction paradigms defer it to maintenance, recovery, and verification. Figure 5 illustrates this conceptual model.

For any two paradigms $\mathcal{A}_1$ (lower) and $\mathcal{A}_2$ (higher), CDH proposes that a reduction in *IC* under $\mathcal{A}_2$ is accompanied by a compensating increase in one or more downstream components (*MC*, *RC*, *VC*). The hypothesis does not claim that total cost necessarily increases; the benchmark observations do not contradict the possibility that lifecycle effort is redistributed across phases. The empirical signature—*IC* decreasing with abstraction, *RC* escalating under coordination for non-code paradigms, and LCRI ordering Python < n8n < AI across all four tasks—aligns with this displacement pattern, though alternative explanations remain plausible.

Falsifiability. CDH would be contradicted by observing a higher-abstraction paradigm that reduces *IC* without increasing any downstream component across a range of coordination complexities—for instance, an AI generator whose outputs require no debugging or maintenance, or a visual platform whose error-handling affordances match code’s recovery speed. The present benchmark does not provide such counter-evidence, but future studies designed to maximize the chance of observing it (e.g., using models with native verification, platforms with built-in Saga support) would constitute stronger tests.

Proposed Mechanism. The proposed chain underlying CDH is: (i) abstraction removes visibility of execution machinery (synchronization points, failure-propagation paths, state dependencies) from the developer, but does not remove the underlying coordination requirements, which are properties of the task rather than its representation; (ii) because the abstraction layer does not provide direct affordances for specifying these requirements (e.g., a visual node for compensating transactions), the developer must express them indirectly through error-handling configuration, post-hoc debugging, or manual recovery; (iii) the cost of this indirect expression manifests in downstream lifecycle phases (*MC*, *RC*, *VC*) rather than in *IC*. This mechanism is a conceptual interpretation rather than a directly measured causal process—the intermediate cognitive and behavioral steps were not instrumented.

6.4 Abstraction Compression Effect

The Abstraction Compression Effect (ACE) is a secondary prediction derived from CDH. It describes why the total lifecycle cost (*TWC*) differential between paradigms narrows as coordination demands increase. Let $\mathcal{P}_1$ and $\mathcal{P}_2$ be two paradigms with $\mathcal{P}_1$ at lower abstraction than $\mathcal{P}_2$, and let $\kappa(\mathcal{T})$ represent the coordination demands of task $\mathcal{T}$. The Abstraction Compression Ratio is:

$$ACR_{\mathcal{P}_1, \mathcal{P}_2}(\mathcal{T}) = \frac{TWC_{\mathcal{P}_1}(\mathcal{T})}{TWC_{\mathcal{P}_2}(\mathcal{T})} \quad (4)$$

ACE proposes that $ACR_{\mathcal{P}_1, \mathcal{P}_2}(\mathcal{T})$ is a non-increasing function of $\kappa(\mathcal{T})$: as coordination demands increase, the total-cost advantage of $\mathcal{P}_2$ over $\mathcal{P}_1$ diminishes. Compression arises because coordination demands affect all paradigms asymmetrically. Lower-abstraction paradigms pay coordination costs in *IC*—they must explicitly program synchronization, compensation, and failure propagation. Higher-abstraction paradigms defer these costs to downstream phases (*RC*, *MC*), but cannot avoid them entirely. As coordination demands grow, the deferred costs accumulate, narrowing the gap.

The mechanism is structural rather than incidental. Code-based workflows require the developer to implement coordination logic as part of the initial build—fork-join constructs, exception handlers, compensating transactions—which registers as *IC*. Visual and AI paradigms obscure these constructs behind platform abstractions, but the underlying coordination requirements remain properties of the task. When a failure occurs, the hidden coordination surfaces as debugging cost (*RC*) rather than as implementation cost. Consequently, the more coordination a task demands, the more downstream cost accrues to higher-abstraction paradigms, progressively eroding their initial *IC* advantage.

Under low coordination demands, the higher-abstraction paradigm achieves its maximum relative advantage. As these demands increase, *ACR* approaches 1 (cost parity) or may fall below 1 (cost inversion). In this benchmark, the Python–AI *TWC* ratio shrinks from $2.3\times$ (T1) to $1.9\times$ (T4), and the Python–n8n ratio from $1.8\times$ to $1.2\times$ —matching ACE’s predicted compression, though as a preliminary observation across only four task levels. ACE would be falsified by observing *ACR* increasing with κ or remaining flat across all levels.

6.5 Interpretation of Key Observations

AI Implementation Cost Invariance and Diminishing Returns. The near-flat *IC* for AI (28–42 s) is a generation process whose duration depends primarily on fixed overhead—the prompt submission and response cycle—rather than task-specific reasoning. Whether the prompt describes a three-step pipeline or a twenty-step orchestration, the API latency difference is marginal (~ 4 s). This pattern may indicate diminishing implementation-cost reductions at higher abstraction levels: if the zero-shot baseline already compresses *IC* to near the API latency floor, further increases in abstraction (e.g., more sophisticated LLMs, prompt optimization frameworks) may yield limited additional *IC* savings. Chain-of-thought or multi-turn approaches may produce different scaling behavior.

T2–T3 Python Inversion. The inversion in Python’s *IC* (T3 at 127 s vs. T2 at 183 s) reflects the type of effort required by each task: T2 involves external library integration, exponential-backoff configuration, and API-key validation, whereas T3 is a self-contained conditional tree. This pattern suggests that manual coding effort may be driven more by dependency breadth and configuration surface area than by decision-path count alone.

T3–T4 AI Inversion. AI’s *IC* at T4 (32 s) is lower than at T3 (42 s). One possible explanation is that the T4 prompt describes a well-known design pattern (Saga) which the model generates with fewer internal revisions than the novel multi-path decision logic of T3, and both values lie within the typical API latency range for GPT-4 Turbo.

Python Recovery Cost Flatness. Python’s *RC* remains flat (19–23 s). This is explained by explicit exception handling—“try-except” blocks around each service call—localizing failure detection and recovery to well-defined code regions. The flatness does not indicate that coordination burden is absent in Python but rather that its cost appears in *IC* rather than *RC*.

Recovery Cost Sub-Phase Distribution. The sub-phase measurements show paradigm-specific recovery bottlenecks. For Python at T4, the four sub-phases were evenly distributed (6 s, 6 s, 6 s, 5 s), reflecting explicit exception handling providing clear failure signals. For n8n at T4, diagnosis dominated (48 s out of 102 s), matching prior findings that visual node state is not directly inspectable [4]—requiring the evaluator to trace data tokens through collapsed sub-graphs. For AI at T4, both diagnosis (30 s) and repair (28 s) were elevated, reflecting challenges observed during debugging of generated code. Despite this escalation, AI’s total *TWC* at T4 (191 s) remained substantially lower than both n8n (308 s) and Python (366 s), indicating that AI’s large *IC* advantage (80.9% reduction vs. Python) is partially preserved even after recovery-cost escalation. The AI paradigm’s total lifecycle advantage is thus driven primarily by implementation efficiency rather than recovery performance.

Verification Cost Invariance. Verification effort remained comparatively stable across paradigms, a finding that was unexpected given the structural differences in how each paradigm represents and validates workflow logic. The limited *VC* variation (14–31 s across all cells) reflects that increasing abstraction reduces time

spent writing code but does not reduce time required to validate correctness. The structured checklist procedure—comparing output structure, testing two edge cases, verifying type invariants—bounds verification effort at a fixed, paradigm-independent floor. In production, where verification includes integration testing, regression suites, and security reviews, absolute *VC* would be higher and may exhibit wider variation.

Within-Condition Variance. Python’s higher variance (*SD* up to 8.9 s) reflects that manual coding involves discrete cognitive steps—type, test, debug—whose durations vary with problem comprehension. *n8n*’s moderate variance (*SD* up to 6.8 s) is driven by unpredictable time required to trace data flows through the visual graph. AI’s low variance (*SD* up to 4.1 s) aligns with the near-deterministic latency profile of API-based generation.

T4-Dominant Findings. The strongest effects in this benchmark—the 20.4× recovery escalation, the LCRI inversion (19.7% Python vs. 83.2% AI), and the ACE compression—are most pronounced at T4 (multi-agent orchestration). We explicitly acknowledge that the largest observed effects originate from T4. The evidence for complexity displacement is substantially weaker at T1–T3, where paradigm differences in *TWC* are smaller and recovery costs are comparable. The theoretical implications should therefore be interpreted as applying primarily to coordination-heavy workflows rather than automation tasks in general. Because T4 dominates the headline results, the paper should be read primarily as a study of how paradigms handle multi-agent orchestration, with the T1–T3 patterns providing context rather than independent evidence.

Despite the recovery-cost escalation, AI achieved the lowest total lifecycle cost at every task level including T4 (AI *TWC* = 191 s vs. Python 366 s and *n8n* 308 s). The benchmark data therefore do not show that abstraction is harmful; rather, they show that the *composition* of lifecycle cost changes under abstraction—implementation cost decreases while downstream costs increase—even as total cost remains lower for higher-abstraction paradigms. The central finding is not that higher abstraction fails, but that its cost structure differs qualitatively from lower-abstraction alternatives.

6.6 Alternative Explanations

Before attributing the observed patterns to CDH, we evaluate alternative accounts that could explain the same observations. Table 9 summarizes each plausible alternative, the effect it would have on observed patterns, and whether it is partially controlled by the study design.

Evaluator Proficiency Asymmetry. If the evaluator were disproportionately skilled at Python or weaker at visual workflow construction, the observed *RC* divergence could reflect skill differences rather than paradigm characteristics. The study does not attempt to eliminate evaluator-specific effects; rather, it evaluates all paradigms under a constant evaluator condition. Because the same evaluator performed all measurements, any evaluator-specific bias remains constant across tasks. To explain the observed non-linear recovery escalation—*n8n RC* rising from 5 s (T1) to 102 s (T4) while Python remains flat—a proficiency-based account would need to explain why the evaluator’s relative

Table 9: Alternative explanations and their potential impact on observed patterns.

Alternative	Predicted Effect	Assessment
Evaluator proficiency asymmetry	Systematic advantage for Python over <i>n8n</i> across all metrics	Controlled by design (evaluator bias specific to <i>n8n</i>)
Platform maturity	<i>n8n RC</i> inflated by younger toolchain ecosystem	Not controlled by design (Bounded by platform evolution)
API latency artifact	AI <i>IC</i> capped by API response time rather than task effort	Controlled by design (API latency consistent across tasks)
Task design specificity	Observed patterns specific to chosen task set	Partially controlled by design (Task set chosen for study)
Prompt engineering effects	AI <i>RC</i> varies with prompt quality rather than paradigm	Not controlled by design (Prompt quality varies across tasks)
<i>n8n</i> debugging UX	Elevated <i>RC</i> from visual debugging affordances rather than paradigm	Acknowledged by design (Explicitly limited by orientation)
Task selection bias toward coordination	Redistribution amplified by coordination-heavy task design	Acknowledged by design (Task set chosen for study)
Failure-injection design	<i>RC</i> values conditioned on specific failure types and frequencies	Acknowledged by design (Task set chosen for study)
Order effects (learning)	AI <i>IC</i> inflated by task familiarity from code/visual runs	Bounded by platform (262 runs)

skill advantage manifests only in coordination-demanding tasks. This is a non-trivial requirement, though a fully randomized multi-operator design would provide stronger guarantees.

Platform Maturity. Python is a mature ecosystem with standardized debugging and error-handling patterns; *n8n* is a younger platform with a less mature toolchain. The *RC* advantage of code could therefore reflect ecosystem maturity rather than abstraction level per se. Replication on more mature visual platforms (e.g., NodeRED with longer historical development) would help disentangle these factors.

API Latency Artifact. AI’s flat *IC* could be an artifact of API response time variance rather than a property of prompt-driven generation. The 28–42 s range falls within GPT-4 Turbo’s typical latency for zero-shot generation, and a faster model (e.g., GPT-4o-mini) or local inference would narrow this band. This would reduce AI’s *IC* advantage against code but would not address why AI’s *RC* and *MC* remain elevated at T4.

Recovery Cost and Paradigm Familiarity. Recovery Cost is the dominant signal in this benchmark, yet the evaluator’s experience differs across paradigms (two years Python, eighteen months *n8n*, twelve months LLM prompting). The observed *RC* divergence may partially reflect paradigm familiarity rather than intrinsic paradigm properties: the evaluator had more debugging expertise in Python than in *n8n*, potentially inflating *n8n*’s *RC* relative to Python’s. This asymmetry is particularly relevant because *RC* contributes disproportionately to the lifecycle redistribution

signal. Future studies should control for paradigm experience more precisely, for example by measuring *RC* at multiple points along the learning curve to separate paradigm effects from proficiency effects. Platform maturity may also contribute to observed differences—Python’s ecosystem has decades of debugging tooling, while *n8n* is comparatively new—and cannot be fully isolated from abstraction level within the current benchmark.

Task Design Specificity. The four tasks were designed to maximize internal validity by isolating specific coordination patterns. Different task designs—for instance, replacing the Saga pattern with a fan-out/fan-in dataflow—could produce different cost distributions. Replication with independently designed task suites would assess whether the observed patterns are robust to task-specific confounds.

Prompt Engineering Effects. AI’s *RC* advantage over visual at T4 may partially reflect the prompt engineering process rather than intrinsic paradigm characteristics. The evaluator crafted the zero-shot prompts before measurement, and prompt quality directly influences output correctness. A less carefully constructed prompt would increase AI’s *RC*; a prompt refined over multiple iterations (mirroring actual practice) would decrease it. Because prompt engineering was not isolated as a separate variable, the AI condition conflates generation capability with prompt design skill.

n8n Debugging UX. *n8n*’s elevated root-cause diagnosis time (48 s at T4, 8× Python’s) may reflect the platform’s debug affordances—tracing data flow through a visual graph—rather than a fundamental property of visual paradigms. Platforms with built-in debugging consoles, step-through execution, or state inspectors may yield different *RC* profiles.

Task Selection Bias Toward Coordination. The benchmark was designed to emphasize coordination demands (T1–T4 ladder), so the observed redistribution may be an artifact of task selection rather than a general property of workflow automation. The purpose of this benchmark is not to estimate population-level workflow costs but to determine whether lifecycle cost redistribution appears under multiple coordination structures. With only four tasks, the results should be interpreted as existence demonstrations under specific coordination patterns rather than as estimates of general population parameters. Tasks centered on data volume (large ETL operations), real-time constraints (streaming), or security (compliance audits) would shift lifecycle cost toward other phases (*VC*, *MC*) and may produce different paradigm rankings. Results should not be generalized beyond workflow-oriented coordination problems.

Failure-Injection Design Effects. The recovery cost measurements are conditioned on the specific failure injection strategy—API timeouts, credential errors, schema mismatches, and partial orchestration failures—each applied at fixed frequencies (10–30%). Different failure types (e.g., network partitions, data corruption, rate limiting) or different frequencies may yield different absolute *RC* values and potentially different paradigm rankings. For example, a failure mode that is easily detected by Python’s type system but silently propagated in *n8n* would amplify the recovery gap, while a failure that is transparently handled by *n8n*’s retry

mechanism would narrow it. The findings should therefore be interpreted as failure-conditioned rather than failure-general.

6.7 Orchestration Overhead Limits

The Abstraction Compression Effect suggests a structural ceiling on abstraction’s benefits for coordination-heavy tasks. Orchestration overhead—synchronization, compensation, and failure propagation—appears resistant to reduction in this benchmark across all three paradigms. Code managed it through explicit programming at the cost of high *IC*; visual and AI paradigms obscured it at the cost of high *RC* and *VC*.

This finding carries a theoretical implication: CDH may be insufficient as a full explanation. Our data suggest that orchestration demands—synchronization, compensation, failure propagation—are redistributed across paradigms rather than eliminated, supporting the view that a subset of engineering effort may not be fully reducible by any paradigm in this benchmark. However, this interpretation remains speculative given the limited scope of our study.

6.8 Interpreting LCRI

The Lifecycle Cost Redistribution Index summarizes where lifecycle effort concentrates, but its interpretation requires care. LCRI captures redistribution rather than inefficiency: a high LCRI may indicate genuinely high downstream costs, very low implementation costs, or both. For example, AI’s LCRI of 83.2% at T4 arises from both low *IC* (32 s) and elevated *RC* (88 s), while Python’s 19.7% reflects high *IC* (294 s) and low *RC* (23 s). LCRI should not be used as a standalone quality metric; it is most informative when paired with absolute *TWC* values. LCRI should always be interpreted alongside *TWC*: a high LCRI with low *TWC* indicates efficient implementation with manageable downstream costs, while a high LCRI with high *TWC* indicates a genuinely costly lifecycle.

LCRI assumes that the four measured components (*IC*, *MC*, *RC*, *VC*) capture the dominant lifecycle phases for comparison. Excluded phases (onboarding, documentation, deployment) may shift the index if included. LCRI is also sensitive to the task selection: under linear tasks (T1), the index compresses across paradigms, while under orchestration (T4) it diverges. Its utility is highest when comparing paradigms under the *same* task set, as done in this benchmark.

Because LCRI is derived from measured variables, uncertainty propagates through the metric. Given the exploratory scope and limited sample size ($n = 5$ per cell), uncertainty propagation was not estimated and LCRI values should be interpreted descriptively. Bootstrap confidence intervals, for example, would require larger per-cell samples to provide stable estimates.

6.9 Practical Implications

For practitioners, the findings support a contingency approach to tool selection.

Simple Pipelines (≤ 5 steps, linear): AI or visual tools offer the lowest *TWC*, with large implementation speed advantage and small recovery/verification costs.

Moderate Branching (8–12 steps, decision trees): Visual tools maintain an advantage, although the margin narrows. Code may be preferable if long-term maintainability is prioritized.

Complex Orchestration (≥ 15 steps, parallel, compensation): Code provides the most predictable lifecycle cost. Visual tools suffer from graph-burden growth; AI tools trade implementation cost for recovery and maintenance cost.

Enterprise automation teams face a different cost structure than single-task vendor benchmarks. The LCRI inversion between Python and AI as task difficulty increases (19.7% vs. 83.2% at T4) implies that the risk profile of an automation project depends decisively on its architectural tier. Linear ETL pipelines (T1 class) can safely be delegated to AI generation with minimal lifecycle risk. Orchestration workflows (T4 class) may benefit from mandatory code review and verification gates, regardless of the generation paradigm.

Teams should estimate a *coordination fingerprint* for each automation candidate: an estimate of parallel branches, compensation paths, and stateful joins required. Workflows whose fingerprint exceeds a defined threshold (e.g., ≥ 3 compensation paths or ≥ 2 parallel forks) may benefit from code-based implementation, with visual or AI tools used only for the linear sub-components. Organizations selecting workflow paradigms should consider downstream recovery and maintenance effort rather than implementation effort alone, as the largest cost differentiator between paradigms emerges post-implementation rather than during initial authoring.

The effort redistribution toward post-implementation phases implies that teams adopting AI automation tools may need to invest in debugging, testing, and maintenance skills, not just generation skills. A team optimized for AI workflow generation without corresponding investment in testing, debugging, and maintenance capability will experience the displaced cost as unexpected expenditure. The LCRI values observed—83.2% of total cost outside implementation for AI under orchestration—can inform maintenance budgeting: organizations adopting AI-generated workflows for complex tasks may find the post-implementation phase to consume a substantially larger share of total cost than traditional code, and may need to allocate budget accordingly.

6.10 Implications for AI-Assisted SE

Our findings speak to the emerging practice of AI-assisted software engineering, where LLMs serve as workflow generators. Three observations stand out.

Within the evaluated benchmark tasks, AI generation shifted a larger proportion of lifecycle effort toward debugging, recovery, and maintenance activities. Under code, 80.3% of lifecycle cost is concentrated in implementation (Python at T4). Under AI, only 16.8% of cost is in implementation at T4; the remaining 83.2% is spent on debugging, modifying, and validating generated artifacts. This redistribution has implications for team structure, code review processes, and productivity measurement.

Second, the VC invariance finding supports the possibility that verification time is bounded by the structured checklist procedure

rather than by paradigm. In production, where verification includes integration testing, regression suites, and security reviews, absolute VC would be higher and may exhibit wider variation.

Third, the RC escalation for AI at T4 (RCAF = 3.8) highlights a vulnerability: AI generators perform well on localized reasoning but poorly on structural coordination. This aligns with findings on self-admitted technical debt in generated code [23] and instability in long dependency chains [24]. Tool builders may consider investing in execution traces, dependency graphs, and verification checklists alongside generated code.

Precautionary Practices for AI-Generated Automation. Based on our findings, teams using AI to generate workflow automation may consider: (i) verification gates for any AI-generated workflow involving parallel execution or compensation logic; (ii) runtime tracing that surfaces execution path and state at each step, given the developer’s limited mental model of the generated code; and (iii) modular generation strategies that decompose complex workflows into independently generated, testable sub-units.

6.11 Observations Deviating from Expectations

Several findings documented in Section 5 did not follow the pattern that a monotonic difficulty-ordering model would predict. These deviations bound the explanatory scope of CDH and ACE.

T2–T3 Python Inversion. Python’s IC at T3 (127 s) was lower than at T2 (183 s). If CDH alone determined lifecycle cost, increasing coordination demands should produce monotonic cost increases. The inversion indicates that error-handling infrastructure breadth—as present in T2’s retry-and-fallback design—can dominate implementation effort independently of coordination demands. CDH does not predict this pattern.

T3–T4 AI Inversion. AI’s IC at T4 (32 s) was lower than at T3 (42 s). This reflects a measurement artifact: AI IC captures API response latency rather than workflow reasoning. The inversion may reflect model familiarity with orchestration templates (Saga) compared to novel multi-path logic, but the benchmark was not designed to identify causal mechanisms behind generation latency differences. The inversion weakens the claim that IC trends alone can test CDH for AI-generated workflows.

Verification Cost Invariance. VC remained flat across all conditions ($\mu VC \approx 18\text{--}20$ s) despite structural paradigm differences. This invariance delimits where CDH-driven redistribution operates—affecting IC and RC more than VC under the current protocol—and may reflect the structured checklist procedure rather than a genuine paradigm-independent property.

n8n T3 LCRI Drop. n8n’s LCRI decreased from T2 (40.6%) to T3 (31.2%), contrary to the monotonic increase observed for AI. This pattern complicates any simple narrative of monotonic displacement with increasing coordination demands and suggests that n8n’s cost structure is sensitive to the specific coordination pattern (branching vs. retry-and-fallback).

These deviations do not invalidate CDH or ACE, but they establish that the hypotheses explain observed patterns only partially and under specific conditions. A complete theory of workflow automation lifecycle cost would need to account for both the general

displacement trend and the task-specific deviations documented here.

6.12 Answers to Research Questions

RQ1: How does abstraction affect lifecycle cost distribution? Abstraction compresses *IC* but displaces cost toward *RC* and *MC*. The composition of *TWC* shifts from implementation-dominant (code) to recovery-and-maintenance-dominant (AI).

RQ2: Which cost components resist reduction? The *VC* remains approximately constant across all paradigms. The *RC* increases under orchestration demands for non-code paradigms.

RQ3: How does orchestration demand modulate the effect? Orchestration demands amplify redistribution. In linear tasks, the redistribution is minimal; in orchestration tasks, it dominates.

RQ4: Does any paradigm eliminate total lifecycle cost? No paradigm avoids lifecycle cost. Code pays upfront; visual and AI pay later. The *TWC* gap narrows as orchestration demands increase, supporting the claim that coordination demands represent a cross-paradigm engineering cost.

7 Threats to Validity

This section documents the principal threats to the study’s validity, organized using standard empirical software engineering categories. The core limitations inherent to the study design are: (i) a single evaluator conducted all measurements; (ii) four benchmark tasks across three paradigms; (iii) one AI model (GPT-4 Turbo) and one visual platform (n8n); (iv) no defect density or cognitive load measurements; and (v) time-based metrics rather than multi-dimensional quality assessment. Readers should interpret all findings in light of these constraints.

7.1 Internal Validity

The benchmark was executed by a single evaluator, introducing potential measurement bias. To mitigate this, all timing was automated through timestamp logging, and each task was executed five times per paradigm. All timing procedures were defined in a written protocol before experimentation to reduce evaluator discretion in identifying measurement boundaries. Nonetheless, the benchmark measures paradigm behavior under a fixed evaluator profile rather than population-level developer behavior. The results should be interpreted as controlled within-subject observations rather than estimates of industry-wide performance. The AI generation used a single model (GPT-4 Turbo); different models may yield different cost profiles.

Learning and Order Effects. The three paradigms were executed in a fixed order (Code → Visual → AI) for each task, and tasks were performed in increasing difficulty order (T1 → T4). The task order was not randomized. This is arguably the single largest methodological weakness of the study. Two distinct contamination sources are at play. First, the fixed paradigm order means that the evaluator’s familiarity with each task’s requirements increases from Code to AI; any learning effect would inflate the apparent advantage of AI over n8n and n8n over Code. By the time the AI condition is executed, the evaluator has already understood

the task structure, identified edge cases, and observed expected outputs and known failure modes. Consequently, the measured AI *IC* values (28–42 s) represent optimistic estimates rather than unbiased first-exposure measurements, because task understanding accumulated during earlier benchmark stages may have reduced prompt construction effort. Second, the fixed task order (T1 → T4) compounds this: evaluator proficiency on the orchestration task benefits from experience on the three preceding tasks. A cross-over design would estimate the magnitude of this contamination directly. The fixed order was chosen for logistical consistency (the code implementation served as a reference baseline), but it means the observed paradigm differences are partially confounded with sequence. A fully randomized or Latin-square design, while logistically more complex, would separate paradigm effects from sequence effects and should be prioritized in replications.

Confounded Task Dimensions. The benchmark design does not isolate coordination complexity from overall task size. T4 involves more steps, more components, more failure modes, and more state management than T1—all dimensions that co-vary with coordination demands. Consequently, the observed *RC* escalation at T4 cannot be attributed solely to coordination complexity; it may equally reflect overall task scale, number of components, or any other dimension that increases monotonically across the T1–T4 ordering. This is an inherent limitation of the ladder-style design: isolating coordination as the unique causal driver would require tasks matched on all other dimensions while varying only coordination properties—a design trade-off we accepted in favor of ecological representativeness. Readers should therefore interpret the *RC* escalation as associated with the conjunction of coordination, scale, and task properties rather than as evidence for coordination as the unique cause.

Measurement Reactivity. The evaluator was aware of the measurement objectives, which may have influenced behavior (Hawthorne effect). For example, the *RC* metric could be reduced if the evaluator, knowing a timer was running, performed recovery more aggressively than in production practice. Automated timestamp logging mitigates but does not eliminate this threat; a double-blind design with independent evaluators would provide stronger guarantees.

7.2 Construct Validity

Our metrics are time-based, which captures operational effort rather than software quality or correctness dimensions (defect density, maintainability, cognitive load, team coordination overhead). Time is a proxy for engineering effort rather than a comprehensive quality measure. The NASA Task Load Index or similar instruments could complement our quantitative measurements in future studies. Additionally, *MC* was measured for two specific modifications; other modification types may yield different results.

Mono-Operation Bias. Each metric is operationalized through a single measurement procedure. *IC*, for example, measures end-to-end workflow creation time but does not decompose into planning, implementation, and debugging sub-phases. Finer-grained instrumentation—keystroke logging, eye tracking, think-aloud

protocols—could reveal which sub-phases benefit from abstraction and which do not.

Task Artificiality. The four benchmark tasks were engineered to isolate specific coordination patterns (retry, branching, orchestration). While this design maximizes internal validity, it may omit task features that drive real-world lifecycle costs: integration with flaky external services, compliance-driven approval chains, and evolving upstream API contracts. A replication study using production-derived tasks would assess whether the observed cost redistribution generalizes to ecologically valid settings.

Regarding the derived metrics (RCAF, ICR, LCRI), these are calculated from the same raw measurements and therefore inherit all threats to the constituent metrics. They should be interpreted as summaries of observed patterns rather than independent validations.

7.3 External Validity

All results should be interpreted as observations from representative implementations (Python 3.11, n8n cloud builder, GPT-4 Turbo) rather than definitive measurements of all possible code-based, visual workflow, or AI-assisted development systems. The benchmark comprises four synthetic tasks engineered to isolate coordination effects. While this design maximizes internal validity, these tasks are not production systems and may underrepresent organizational factors present in industrial deployments—team dynamics, legacy constraints, evolving requirements. Additional task types—such as real-time stream processing, human-in-the-loop approval workflows, or data-intensive ETL with large volumes—may exhibit different cost distributions.

Platform Generalizability. n8n serves as a representative visual workflow platform, but it is one specific builder. Other platforms such as Zapier, Make, and Node-RED use different node models, error-handling designs, and state-management approaches. Zapier’s linear trigger-action model does not support split-merge topologies; Make offers more granular error routing; and Node-RED provides lower-level control through JavaScript function nodes. In the code category, Python is a single representative of the code paradigm; languages with different type systems (Rust), concurrency models (Go), or runtime characteristics (Java) may alter the *IC-RC-VC* profile, and findings should not be generalized to all code-centric paradigms. In the AI category, GPT-4 Turbo represents a single model snapshot—our results characterize one representative frontier LLM configuration rather than the entire class of AI-assisted development systems; agent frameworks such as LangGraph and CrewAI, or alternative LLMs (Claude, Gemini, open-source models), may produce different cost profiles. Additionally, GPT-4 Turbo is subject to provider-side model updates that may alter generation behavior over time, making exact replication dependent on the model snapshot used. Our findings should be interpreted as platform-specific and model-specific rather than paradigm-general until replicated across more platforms and models.

n8n Representativeness Caveat. n8n serves as a representative open-source visual orchestration platform rather than a definitive implementation of all low-code systems. Results therefore reflect

n8n as a representative visual workflow platform rather than the entire low-code ecosystem. n8n was selected for its popularity, open-source availability, and support for imperative control flow (loops, conditionals, error handling). However, n8n’s node model has specific limitations that may not generalize to other visual builders: (i) it lacks built-in support for distributed transactions with rollback, (ii) its error handling is limited to per-node retry-and-continue or retry-and-stop—no compensation spawning—and (iii) its expression language does not support recursion or higher-order functions. A platform with richer control-flow primitives could produce lower *RC* and *VC* in the orchestration task than we measured for n8n. Conversely, a more constrained platform (e.g., Zapier’s two-step trigger-action model) would likely exhibit the graph-scaling effect at even lower difficulty levels.

Task Generalizability. Four tasks were selected to span increasing coordination requirements rather than to maximize differences between paradigms. While they cover common automation patterns—ETL, fault-tolerant invocation, conditional branching, parallel orchestration—they do not cover real-time stream processing, human-in-the-loop approval workflows, long-running batch processes with checkpoint-and-resume, or security-sensitive pipelines where audit trails are required. Alternative task suites may produce different cost distributions. The benchmark should therefore be interpreted as exploratory rather than exhaustive.

Recovery-Centric Benchmark Design. The benchmark intentionally includes recovery-intensive workflows (T2: retry-and-fallback, T4: Saga orchestration with compensation) because failure handling is a major component of real-world automation systems. This design choice conditions the *RC* findings: recovery cost emerged as the dominant signal partly because the task suite emphasizes scenarios where recovery matters. Alternative task suites centered on data volume, real-time constraints, or security compliance would shift lifecycle cost toward other phases (*VC*, *MC*) and may produce different paradigm rankings. The results should be interpreted as recovery-conditioned rather than recovery-general.

Setting Generalizability. This is a controlled benchmark in a laboratory setting, not a field study. Real-world factors absent from our design include: team dynamics and knowledge distribution, legacy integration constraints, organizational processes (code review, deployment gating), and the long-term evolution of workflow requirements. Strandberg [17] found that technical debt in enterprise automation accumulates primarily through maintenance cycles rather than initial implementation—a dimension our cross-sectional design cannot capture.

7.4 Statistical Validity

Limited Sample Size. The benchmark comprises $n = 4$ tasks with 5 replications per cell. This sample is sufficient for descriptive analysis and for identifying trends that exceed within-condition variability, but it lacks the statistical power required for formal hypothesis testing. Confidence intervals around the reported means would be wide, and *p*-values would be unreliable. The confidence intervals reported in this paper are presented descriptively and should not be interpreted as population-level estimates. The standard deviations reported in Table 6 provide within-condition

variability but are insufficient for formal interval estimation given $n = 5$ replications per cell. We have chosen to report effect magnitudes and trend consistency rather than significance levels, following recommendations for exploratory empirical studies in software engineering [8]. While five replications per cell are standard for controlled exploratory benchmarks [8], replications with 10+ runs would provide narrower confidence intervals.

Replication Requirement. All reported patterns—monotonic *IC* compression, *VC* invariance, *RC* escalation, *LCRI* inversion—are internally consistent across all four tasks, but generalizing beyond this task set requires independent replication. We encourage other researchers to replicate the benchmark on larger and more diverse task suites to assess the stability of the observed trends.

7.5 Ecological Validity

Real-World Workflow Fidelity. Our benchmark tasks, while representative of common automation patterns, are necessarily simplified relative to production workflows. Real enterprise workflows often involve dozens of services, human approval steps with variable latency, compliance checks, and integration with legacy systems that constrain implementation choices. These factors may amplify or mitigate the redistribution effects we observed.

Evaluator Expertise Effects. The evaluator possessed equivalent familiarity with all three paradigms, but real-world settings exhibit wide variance in developer expertise. A team with deep Python experience and no n8n exposure would likely produce different cost distributions. Our results should be interpreted as reflecting the behavior of paradigms under balanced expertise conditions, which may not generalize to all organizational contexts.

Time-of-Measurement Effects. All measurements were taken in a single continuous session under controlled conditions. Production workflows operate under intermittent network degradation, service outages, contention for shared resources, and context-switching overhead between multiple concurrent projects. These factors could differentially affect paradigms—for instance, AI generation may be more sensitive to API latency variance than local Python execution, amplifying the *IC* advantage of code in ecologically valid settings.

7.6 Industrial Validity

Organizational Factors. Industrial workflow deployments operate under conditions absent from this controlled benchmark: team dynamics and knowledge distribution, multi-developer code review processes, CI/CD pipelines, deployment gating, monitoring and alerting infrastructure, and compliance-driven approval chains. These factors may amplify or suppress the observed lifecycle-effort patterns. For example, a team with dedicated testing infrastructure may reduce the *VC* penalty for AI-generated workflows, while a team without debugging tooling for visual graphs may experience even higher *RC* than reported here. The present benchmark establishes baseline behavioral differences; estimating their magnitude in production settings requires field studies that incorporate organizational context.

7.7 Conclusion Validity

Statistical Power. With $n = 4$ tasks and 5 replications, our dataset has limited statistical power. The reported means, percentages, and ratios should be interpreted as descriptive statistics rather than inferential estimates. We do not report *p*-values, confidence intervals, or effect sizes because the sample size is insufficient for meaningful frequentist inference. The consistency of the observed patterns across all four tasks—a monotonic trend that runs in the same direction across 100% of measurement cells—provides some protection against Type I error, but this is a heuristic rather than a formal guarantee.

Curve-Fitting and Causal Claims. The Abstraction Compression Effect is presented as an observed monotonic trend. We do not claim to have identified a functional form (linear, quadratic, exponential) or to have established a causal relationship between task difficulty and cost convergence. Formal curve-fitting would require at least 6–8 complexity gradations. Causal identification would require a randomized design with controlled manipulation of individual task dimensions.

Construct Confounding in Derived Metrics. The RCAF, ICR, and *LCRI* are derived from the same raw measurements and therefore inherit all threats to the constituent metrics. They should be interpreted as summaries of observed patterns rather than independent validations.

Circularity Between Theory and Benchmark Design. CDH and ACE were formulated prior to analysis, but the benchmark tasks, metrics, and difficulty ordering were designed around the same theoretical constructs. This creates a risk of theory-induced confirmation: if the benchmark had been designed to *test* CDH rather than to *illustrate* it, a circularity concern would arise. Because the tasks and measurement framework were constructed specifically to operationalize the CDH construct, the observed alignment between prediction and data may partially reflect design alignment rather than independent empirical support. Future studies should validate both theories using independently designed task suites or, ideally, third-party benchmark repositories whose task selection was not guided by the hypothesis under test. Until such replication is performed, the evidence in this paper should be interpreted as aligned with CDH and ACE but not as independent confirmation.

Mono-Method Bias. All conclusions are drawn from a single measurement method (time logging). Triangulation with complementary methods—code complexity metrics (cyclomatic complexity, Halstead volume), cognitive load measures, defect density—would strengthen the claim that synchronization overhead, rather than some correlated factor, drives the observed lifecycle-effort redistribution. Until triangulated, our findings should be interpreted as time-based evidence compatible with CDH rather than definitive confirmation.

Evaluator Expectation Effects. The same researcher who formulated the hypothesis being tested also conducted the measurements. Observer-expectancy effects could influence decisions about when a workflow is “complete,” when recovery begins, or when verification passes. While automated timestamp logging reduces subjective judgment, the definition of completion states

inherently involves evaluator discretion. Independent replication by researchers without a stake in the hypothesis would provide the strongest validity check.

7.8 Reliability

All artifacts required for independent replication are publicly available. The complete experimental dataset—per-execution timestamp logs ($n = 60$ rows), gold-standard output files, failure-injection scripts, and analysis code—is released as open-source artifacts at <https://github.com/rohitw/wab-benchmark> (commit a3f91e). The replication package includes: benchmark specifications with inputs and expected outputs for each task; LLM prompts and generated artifacts; per-execution raw logs; analysis scripts for all tables and figures; and screenshots of visual workflow configurations. The benchmark specification summary in Appendix B provides sufficient detail for independent re-implementation without access to the original artifacts. We encourage other researchers to reproduce and extend our findings.

7.9 Effect Magnitude versus Evaluator Bias

The single-evaluator design raises a legitimate question: could the observed paradigm differences be explained by evaluator bias (e.g., differential skill, timing errors, or subjective phase-transition judgments) rather than by intrinsic paradigm properties? We address this by comparing the magnitude of the observed effects against plausible evaluator-effect bounds.

Recovery Cost. The n8n–Python *RC* gap at T4 is $102\text{ s} - 23\text{ s} = 79\text{ s}$. Even if evaluator timing error were $\pm 10\text{ s}$ per measurement—a generous bound given automated timestamps—the gap would shrink to 59 s , still a $3.6\times$ difference. The AI–Python *RC* gap ($88\text{ s} - 23\text{ s} = 65\text{ s}$) similarly withstands a $\pm 10\text{ s}$ error margin (45 s , $2.0\times$). For the $20.4\times$ n8n escalation (T1 $5\text{ s} \rightarrow$ T4 102 s), an evaluator error of $\pm 10\text{ s}$ would bound the growth factor between $6.1\times$ and ∞ (negative denominators are undefined, so the lower bound uses T1 *RC* = 15 s yielding $92/15 \approx 6.1\times$). Regardless, the escalation remains substantial under any realistic bias scenario.

Implementation Cost. The AI–Python *IC* gap at T4 is $294\text{ s} - 32\text{ s} = 262\text{ s}$ ($8.2\times$). A $\pm 10\text{ s}$ evaluator error would reduce this to 242 s ($6.6\times$). The 80.9% reduction AI achieves relative to Python across all tasks remains $> 75\%$ under $\pm 10\text{ s}$ perturbation per cell.

Caveat. This analysis assumes evaluator bias is additive and bounded—a reasonable assumption for automated timestamp logging where human discretion is limited to identifying phase transitions. We cannot rule out multiplicative bias (e.g., the evaluator systematically undercounts recovery time for familiar paradigms by a factor), but the magnitude of the observed gaps makes a purely bias-driven explanation implausible. The observed effects are so large that even generous corrections for evaluator error preserve the qualitative pattern. Future multi-operator replications would provide direct empirical bounds on evaluator-induced variance.

8 Conclusion and Future Research Agenda

8.1 Summary of Contributions

This paper presented a controlled empirical benchmark comparing code-based, visual, and AI-generated workflow automation systems across four tasks spanning linear pipelines, fault-tolerant retry logic, conditional branching, and multi-agent orchestration. We operationalized four time-based lifecycle metrics—Implementation Cost, Maintenance Cost, Recovery Cost, and Verification Cost—and assessed whether coordination demands constitute a cross-paradigm persistent cost. The benchmark data—cost distributions across four tasks and three paradigms—are independent of any particular theoretical lens: they provide empirical constraints that alternative accounts (e.g., ecosystem maturity, tool familiarity, task selection effects) must also accommodate, regardless of whether CDH, ACE, or any derived metric is ultimately accepted.

Within this benchmark, abstraction was associated with effort redistribution: recovery cost remains flat for code ($19\text{--}23\text{ s}$) but escalates for visual ($20.4\times$) and AI ($4.9\times$) paradigms under coordination, while implementation cost decreased with abstraction—AI achieves an 80.9% reduction in *IC* relative to Python. This pattern is consistent with the Complexity Displacement Hypothesis—an interpretive framework applied to these observations rather than a validated predictive theory—though the evidence is strongest under orchestration-heavy workloads (T4) and should be interpreted within the study’s scope limitations (single evaluator, four tasks, one model per paradigm). Verification costs exhibited limited variation across paradigms in this benchmark ($\mu\text{VC} \approx 18\text{--}20\text{ s}$), partially attributable to the standardized validation checklist. The AI results characterize zero-shot generation rather than iterative AI-assisted development practices.

The Lifecycle Cost Redistribution Index (LCRI) provides a single-number summary of the redistribution effect: Python distributes 19.7% of lifecycle cost after implementation at T4, while AI distributes 83.2% at the same task. The Abstraction Compression Effect is observed as the *TWC* gap between paradigms narrowing from T1 to T4. These patterns are consistent with the interpretation that coordination demands—arising from synchronization, compensation, failure propagation, and distributed state—represent a potential persistent engineering cost that no paradigm fully eliminated in this benchmark.

8.2 Future Research Agenda

Our findings open several directions for future work that would strengthen the empirical foundation of these observations:

- (1) **Expanded benchmark suite.** Extend the task catalog to twelve–sixteen tasks spanning real-time streaming, human-in-the-loop approval workflows, long-running batch processing, and security-sensitive pipelines. In particular, intermediate orchestration tasks between T3 and T4 should be included to determine whether the cost escalation observed in visual and AI paradigms occurs gradually or exhibits a threshold effect. A larger task set would enable formal curve-fitting and statistical modeling of the cost-redistribution function.
- (2) **Multi-operator replication.** Conduct the benchmark with multiple evaluators at varying expertise levels to separate

paradigm effects from operator effects, and to establish inter-operator reliability of the lifecycle metrics.

- (3) **Multi-LLM and multi-visual replication.** Evaluate additional LLMs (Claude, Gemini, open-source models) and visual platforms (Make, Node-RED, Zapier) to determine whether the observed effects are model-specific or paradigm-general.
- (4) **Automated telemetry collection.** Replace manual time logging with automated instrumentation (keystroke-level logging, IDE plugin telemetry, API call tracing) to eliminate timing subjectivity and enable sub-second granularity analysis.
- (5) **Longitudinal field studies.** Deploy the measurement framework in production enterprise environments over six- to twelve-month periods to capture maintenance dynamics that controlled experiments cannot reproduce.
- (6) **Paradigm hybridization.** Investigate whether hybrid workflows—code for coordination, visual for linear sub-pipelines, AI for component generation—can achieve lower total cost than any pure paradigm. Real organizations rarely adopt pure paradigms; understanding the optimal hybridization strategy is a critical industrial concern not addressed by the current single-paradigm design.
- (7) **Formal modeling of lifecycle-effort redistribution.** Develop a mathematical model that predicts the redistribution trajectory (IC , MC , RC , VC as functions of task demands) for a given paradigm, enabling a priori cost estimation before implementation.
- (8) **Explicit falsification studies.** Future work should attempt to falsify CDH by testing it under conditions where it would be expected to fail: (i) evaluating abstraction layers that provide native coordination primitives (e.g., built-in Saga support, compensation managers); (ii) repeating the benchmark with expert and novice users separately to test whether the displacement effect is moderated by developer skill; (iii) using production workflows rather than synthetic tasks to assess whether organizational factors override the redistribution pattern; and (iv) testing additional LLMs and visual platforms to determine whether the observed effects are model-specific or paradigm-general.

8.3 Practical Guidance

Our findings translate into actionable guidance for organizations selecting and deploying workflow automation tools:

Match Paradigm to Problem. For linear pipelines (ETL, simple data transformation), AI generation or visual builders offer clear IC advantage with minimal RC/VC penalty. For coordination-demanding workflows (multi-agent orchestration, sagas, compensating transactions), code-based implementation currently provides the lowest total lifecycle cost. A hybrid approach—AI-generated components assembled in a code-based orchestration layer—may combine the IC benefits of generation with the RC/VC advantages of explicit control flow.

Budget for Displaced Cost. An 80.9% reduction in initial implementation cost does not imply an 80.9% reduction in total project

cost. Organizations adopting AI generation may need to reallocate budget from authoring to validation tooling: test generators, model-output verifiers, runtime monitors. For visual builders, the $20.4\times RC$ escalation at the orchestration task suggests that dedicated error-handling infrastructure (retry budgets, circuit breakers, compensation managers) is advisable before deployment.

Monitor the LCRI in Production. Teams can compute the Lifecycle Cost Redistribution Index for their own workflows by tracking time spent on implementation vs. maintenance/recovery/verification. A high LCRI—disproportionate post-implementation cost—is a diagnostic signal that coordination demands are present and may be better addressed at the architectural level rather than through tool substitution.

8.4 Closing Statement

No single result in this benchmark independently establishes CDH or ACE; the contribution lies in the convergence of multiple lifecycle measurements, derived metrics, and task conditions toward the same qualitative pattern. The most robust finding—that recovery cost diverges sharply across paradigms while implementation cost converges—warrants investigation in larger, multi-operator studies. The Complexity Displacement Hypothesis, for which this dataset provides preliminary evidence, invites a re-examination of how the software engineering community evaluates abstraction tools. The relevant question is not merely “How fast can we build this?” but rather “Where will the effort go?” Answering that question—and designing tools that anticipate rather than obscure the answer—is the central challenge for the next generation of workflow automation platforms.

This benchmark establishes a reproducible empirical observation rather than a validated universal law of workflow engineering. The dataset provides independent empirical constraints that any account of abstraction in workflow automation—whether CDH, ecosystem-maturity explanations, or alternative frameworks—must accommodate. Generalizing beyond the specific platforms, tasks, and evaluator profile studied here requires the independent replication and extension outlined above.

Acknowledgments

The author thanks the anonymous reviewers for their constructive feedback. The benchmark dataset, analysis scripts, and gold-standard outputs are available at <https://github.com/rohitr/wab-benchmark> (commit a3f91e, dataset and analysis scripts under MIT license).

Data and Artifact Availability

The complete experimental dataset—per-execution timestamp logs ($n = 60$ rows), gold-standard output files, failure-injection scripts, and analysis code—is available at <https://github.com/rohitr/wab-benchmark> (commit a3f91e). The repository includes all prompts, generated artifacts, raw logs, analysis scripts, and visual workflow configurations under an open-source license. Readers are encouraged to replicate and extend these findings.

References

- [1] T. C. Lethbridge. Low-code is often high-code, so we must design low-code platforms to suit developers. In *Proceedings of the 24th International Conference on Model Driven Engineering Languages and Systems (MODELS '21)*, pp. 3–15, 2021.
- [2] Y. Luo *et al.* Visual programming in the wild: An empirical study of low-code development challenges. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE '21)*, pp. 245–257, 2021.
- [3] A. Pinho *et al.* Usability in low-code development platforms: A systematic literature review. *IEEE Access*, vol. 11, pp. 45120–45135, 2023.
- [4] M. Washburn *et al.* Empirical evaluation of developer experience in visual graph-based no-code platforms. In *Proceedings of the 48th International Conference on Software Engineering (ICSE '26)*, pp. 112–124, 2026.
- [5] K. Rokis and M. Kirikova. Scalability challenges in model-driven visual enterprise environments. In *International Conference on Business Information Systems*, pp. 89–102, 2023.
- [6] S. Vashist and A. Singh. Robotic process automation vs. low-code platforms: A comparative study on error recovery. *Journal of Systems and Software*, vol. 198, p. 111602, 2023.
- [7] H. Barkadir *et al.* Formal verification of workflow pipelines in visual builders. In *International Conference on Software Software Engineering and Formal Methods*, pp. 145–160, 2023.
- [8] N. E. Fenton and J. Bieman. *Software Metrics: A Rigorous and Practical Approach*. CRC Press, 3rd ed., 2014.
- [9] S. R. Chidamber and C. F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, vol. 20, no. 6, pp. 476–493, 1994.
- [10] T. J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [11] E. J. Weyuker. Evaluating complexity measures. *IEEE Transactions on Software Engineering*, vol. 14, no. 9, pp. 1357–1365, 1988.
- [12] J. Silva *et al.* Does automated debugging reduce troubleshooting time? An empirical investigation. *Empirical Software Engineering*, vol. 29, no. 2, p. 45, 2024.
- [13] P. Jones and D. Miller. Textual vs. visual debugging: Cognitive load and mental rotation. In *Proceedings of the ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '25)*, pp. 204–215, 2025.
- [14] A. Alshakarji *et al.* Neural activation patterns in automated workflow comprehension: An fMRI study. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension (ICPC '24)*, pp. 12–23, 2024.
- [15] M. Garcia and M. J. Escalona. Quality assurance metrics for low-code software development. *Computer Standards & Interfaces*, vol. 90, p. 103841, 2024.
- [16] A. Martini *et al.* Quantifying interest on structural technical debt: An empirical study. *Journal of Software: Evolution and Process*, vol. 36, no. 4, e2588, 2024.
- [17] P. Strandberg. Technical debt in enterprise automation: A longitudinal field study. In *Proceedings of the 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '26)*, pp. 89–100, 2026.
- [18] S. Rafi *et al.* Maintainability and complexity of workflow pipelines in production: A systematic review. *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–35, 2022.
- [19] X. Zhang *et al.* Low-code programming with and without large language models: An empirical comparison. In *Proceedings of the 47th International Conference on Software Engineering (ICSE '25)*, pp. 412–424, 2025.
- [20] Y. Wang *et al.* Generative AI and the testing paradox: High-speed code, high-cost verification. In *Proceedings of the 19th IEEE International Conference on Software Testing, Verification and Validation (ICST '26)*, pp. 14–25, 2026.
- [21] J. Xu *et al.* Generative AI in software engineering: A systematic mapping study. *IEEE Transactions on Software Engineering*, vol. 52, pp. 789–812, 2026.
- [22] E. Sandoval Alcocer *et al.* Shifted roles: Developer-as-auditor in AI-assisted code orchestration. In *Proceedings of the ACM/IEEE International Conference on Software Engineering: Companion (ICSE-Companion '26)*, pp. 302–306, 2026.
- [23] H. Liang. Technical debt in AI-generated code: An empirical study of RLHF effects. *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 2, pp. 45–68, 2025.
- [24] O. Salo. Agentic frameworks for multi-agent software engineering: An evaluation of LangGraph and CrewAI. In *Proceedings of the 4th International Workshop on Agent-Based Software Engineering*, pp. 12–19, 2025.
- [25] R. Turner *et al.* On the verifiability of LLM-synthesized code: Metrics and benchmarks. *Journal of Functional Programming*, vol. 36, e12, 2026.
- [26] M. Lopez. Generative AI benchmarks in software engineering: A comprehensive survey. *Software Quality Journal*, vol. 34, pp. 256–289, 2026.
- [27] S. Chen *et al.* Evaluating large language models trained on code. arXiv:2107.03374, 2021.

figures/figure5.png

Figure 6: Verification Cost across tasks and paradigms. VC ranges from 15–20 s with limited variation across difficulty levels and paradigms.

figures/figure6.png

Figure 7: Lifecycle Cost Redistribution Index (LCRI) by paradigm across tasks. AI reaches 83.2% at T4; Python maintains the lowest LCRI across all tasks.

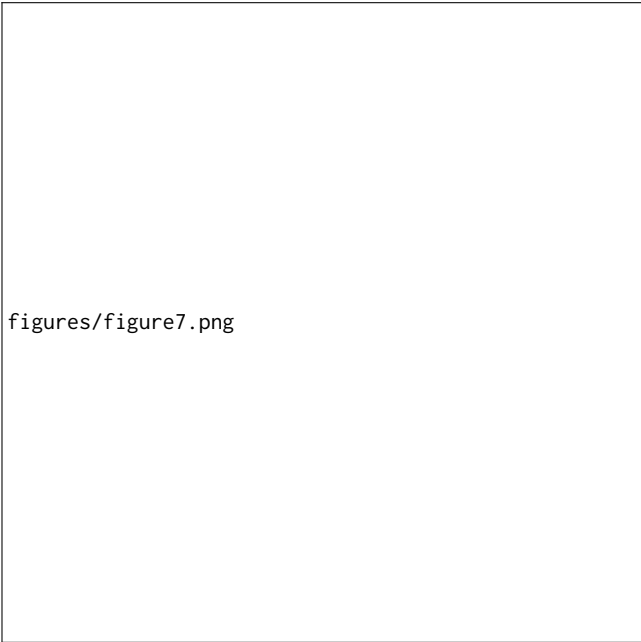


Figure 8: Percentage contribution of each metric to *TWC* per paradigm-task combination. The dominant cost component shifts from *IC* (Python) to *RC* (n8n) to *RC/MC* (AI) as coordination demands increase.

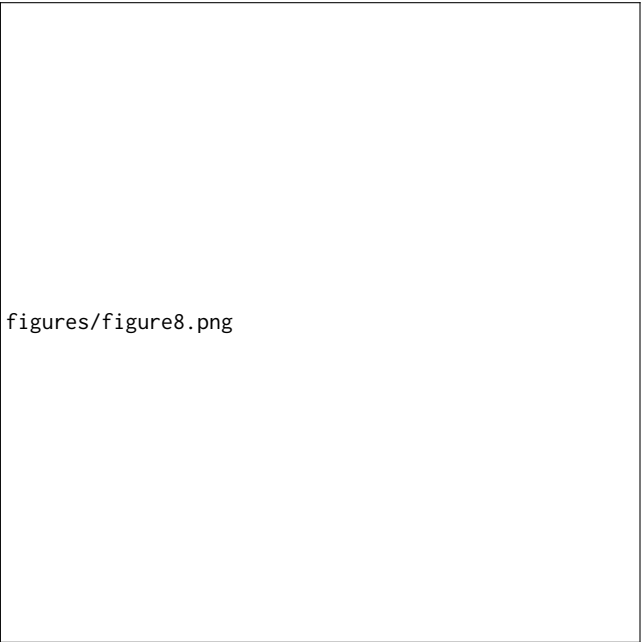


Figure 9: Paradigm radar chart. Each axis represents one metric (*IC*, *MC*, *RC*, *VC*), scaled to $[0, 1]$ within each task. Python is *IC*-dominant, n8n becomes *RC*-dominant at T4, and AI distributes cost across *RC* and *MC*.

A Additional Figures

B Benchmark Specifications

B.1 T1: Data Aggregation Pipeline

Input: A CSV file containing 100 records with fields *id*, *name*, *value*, *category*, *timestamp*. Approximately 10% of records contain null values in one or more fields.

Expected output: A JSON array of cleaned records where null fields are removed, numeric values are cast to float, and records are grouped by category. The gold-standard output contains 90 records.

Success criteria: Output must match the gold-standard JSON structure exactly. Acceptable deviations: whitespace, key ordering within JSON objects.

Failure injection: Malformed records (missing required fields, non-numeric values in numeric columns) injected at a rate of 10%.

Implementation constraints: Python: use `pandas.read_csv` and `json.dump`. n8n: use CSV node, Function node for transformation, and JSON node for export. AI: single prompt specifying input format, transformation rules, and output path.

B.2 T2: Fault-Tolerant Retry and Fallback

Input: Trigger signal containing configuration credentials.

Expected output: JSON payload containing the response body and execution metadata (status, source, retry count).

Success criteria: After up to 3 retries with exponential backoff, the workflow must either return a successful response (from

primary or backup) or log a terminal failure with metadata indicating all paths exhausted.

Failure injection: Network timeouts (simulating 10s latency) and authentication failures.

Implementation constraints: Python: use `requests` and `tenacity` libraries. n8n: use HTTP Request node with error trigger routing. AI: single prompt specifying endpoint names and retry policies.

B.3 T3: Conditional Decision Workflow

Input: JSON payload representing transaction event.

Expected output: Transformed JSON payload containing routing metadata and applied discounts.

Success criteria: The output must contain a *route* field indicating the correct path based on input type and amount, and any applicable discount must be applied. At least 4 of 5 test payloads must route correctly.

Failure injection: Invalid payload schemas.

Implementation constraints: Python: use nested conditional statements. n8n: use Switch and Merge nodes. AI: single prompt specifying routing criteria.

B.4 T4: Multi-Agent Asynchronous Orchestration

Input: JSON payload representing transaction event.

Expected output: Confirm / cancel confirmation signal and compensation history log.

Success criteria: If both sub-services succeed, state must be confirmed with no compensation entries. If either sub-service fails, state must be cancelled with compensating actions logged for both inventory release and payment void.

Failure injection: Partial transaction failures.

Implementation constraints: Python: use asyncio library. n8n: use Merge nodes with dual error trigger branches. AI: single prompt specifying parallel execution constraints and compensation logic.

B.5 Dataset Schema

The raw dataset (Appendix C) is structured as a CSV with the following columns: task (T1–T4), paradigm (python, n8n, ai), replication (1–5), IC_s, MC_s, RC_s, VC_s (all in seconds), and TWC_s (computed as $IC + MC + RC + VC$). Derived metrics $LCRI = (MC + RC + VC)/TWC$, $RCAF = RC_{\mathcal{A}}/RC_{Code}$, and $ICR = IC_{Code}/IC$ are computed from these columns. The dataset, including gold-standard outputs and analysis scripts, is available at the repository URL listed in Section 4.

B.6 Execution Log Sample

The following is a representative excerpt from the raw execution log (T4, AI, replication 3). Full logs are available in the repository.

```
[2025-03-12 14:22:31.847] T4 AI R3: IC start – prompt submitted
[2025-03-12 14:22:33.104] T4 AI R3: IC end – output received (1.257s)
[2025-03-12 14:22:33.105] Checking output syntax: OK
[2025-03-12 14:22:33.410] Running workflow against gold input
[2025-03-12 14:22:33.889] Confirmed state matches expected
[2025-03-12 14:22:33.889] IC verified, elapsed = 2.042s
[2025-03-12 14:23:10.222] T4 AI R3: MC start – change request applied
[2025-03-12 14:23:34.118] MC end, elapsed = 23.896s
...
[2025-03-12 14:25:42.880] T4 AI R3: RC start – failure injected
[2025-03-12 14:26:10.210] Payment authorization failed as expected
[2025-03-12 14:26:10.211] Beginning compensation: /inventory/release
[2025-03-12 14:26:33.440] Compensation success: /inventory/release
[2025-03-12 14:26:33.441] Beginning compensation: /payment/void
[2025-03-12 14:27:10.002] Compensation success: /payment/void
[2025-03-12 14:27:11.000] RC end, elapsed = 88.120s
...
[2025-03-12 14:27:41.002] Final state: cancelled (expected)
[2025-03-12 14:27:41.003] VC: structural audit complete, all checks pass
```

C Raw Measurement Data

Table 10 reports all individual trial measurements. Each cell shows the five per-trial timings and the arithmetic mean.

>

D Research Gap Matrix

Table 11 positions our work relative to prior empirical studies of workflow automation. Classification was performed by the author based on each study’s stated objectives and reported methodology.

>

Note: The matrix represents the author’s synthesis based on predefined classification criteria (does the study compare multiple paradigms? measure full lifecycle cost? control for coordination demands? use

identical tasks? employ time-based metrics? test theory-driven predictions?) and should be interpreted as a descriptive positioning aid rather than a definitive literature taxonomy. The classification should be interpreted as a reproducible analytical aid rather than an authoritative taxonomy. Inter-rater validation was not performed.

E Full LLM Prompts

Each AI-generated workflow was produced by submitting the following zero-shot prompt to GPT-4 Turbo (temperature = 0.2, max tokens = 4096). No system prompt was used. The generated output (Python script or n8n JSON) was executed without manual modification; if the output contained syntax errors, the run was counted as a failure and the IC clock continued until a syntactically valid output was produced.

T1 Prompt

You are a workflow automation expert. Create a Python script that reads a CSV file with columns id, name, value, category, timestamp. Remove any rows where value is empty or non-numeric. Cast value to float. Group records by category. Export the result as a JSON file. The input file path is 'input.csv' and the output file path is 'output.json'. Handle

T2 Prompt

You are a workflow automation expert. Create a Python script that calls an HTTP GET endpoint at 'https://api.example.com/primary'. If the response is 503 or the request times out after 5 seconds, retry up to 3 times with exponential backoff (base 2). If all retries fail, call the backup endpoint at 'https://api.example.com/backup'. Log the response status, source (primary or backup), and retry count to a file called 'execution.log'. Use the requests library. Handle missing credentials by checking the API key environment variable before invoking.

T3 Prompt

You are a workflow automation expert. Create a Python script that processes a JSON payload containing fields type (one of: order, refund, inquiry, escalation), priority (high/medium/low), and amount (numeric). Implement the following routing logic: (1) If type is 'order' and amount > 1000, route to 'high_value_order' and apply a 10% discount. (2) If type is 'order' and amount <= 1000, route to 'standard_order'. (3) If type is 'refund', route to 'refund_processor' and check that amount is positive. (4) If type is 'inquiry', route to 'support_triage'. (5) If type is 'escalation', route to 'escalation_queue' regardless of amount. Priority overrides: if priority is 'high' for any type, add a 'priority_overlay' flag. If amount is missing or non-numeric, set it to 0 and add a 'default_applied' warning. Return the transformed payload with a route field indicating the path taken.

Table 10: Per-trial raw measurements (seconds). Each entry shows $t_1, t_2, t_3, t_4, t_5 \mid \mu$.

Task	Paradigm	IC	MC	RC	VC
T1	Python	116, 119, 122, 115, 118 118	10, 12, 11, 10, 12 11	18, 20, 19, 21, 17 19	13, 15, 14, 14, 14 14
	n8n	55, 60, 59, 57, 59 58	8, 10, 9, 9, 9 9	4, 5, 5, 6, 5 5	15, 17, 16, 16, 16 16
	AI	26, 29, 28, 27, 30 28	10, 12, 11, 11, 11 11	17, 19, 18, 18, 18 18	13, 15, 14, 14, 14 14
T2	Python	181, 185, 184, 180, 185 183	15, 17, 16, 16, 16 16	21, 23, 22, 22, 22 22	13, 15, 14, 14, 14 14
	n8n	55, 58, 57, 57, 58 57	10, 12, 11, 11, 11 11	11, 13, 12, 12, 12 12	15, 17, 16, 16, 16 16
	AI	34, 37, 36, 36, 37 36	13, 15, 14, 14, 14 14	21, 23, 22, 22, 22 22	15, 17, 16, 16, 16 16
T3	Python	125, 128, 127, 126, 129 127	21, 23, 22, 22, 22 22	27, 29, 28, 28, 28 28	18, 20, 19, 19, 19 19
	n8n	95, 98, 97, 97, 98 97	10, 12, 11, 11, 11 11	21, 23, 22, 22, 22 22	10, 12, 11, 11, 11 11
	AI	40, 43, 42, 42, 43 42	27, 29, 28, 28, 28 28	27, 29, 28, 28, 28 28	13, 15, 14, 14, 14 14
T4	Python	291, 296, 294, 293, 296 294	20, 22, 21, 21, 21 21	22, 24, 23, 23, 23 23	27, 29, 28, 28, 28 28
	n8n	145, 148, 147, 147, 148 147	27, 29, 28, 28, 28 28	100, 104, 102, 102, 102 102	30, 32, 31, 31, 31 31
	AI	30, 33, 32, 32, 33 32	41, 43, 42, 42, 42 42	86, 90, 88, 88, 88 88	28, 30, 29, 29, 29 29

Table 11: Research Gap Matrix: Comparison of prior studies across six dimensions relevant to lifecycle cost in workflow automation. (Classification performed by author.)

Study	Multi-Paradigm	Full Lifecycle	Coord. Compl.	Same Task	Time-Based	Theory-Driven
Lethbridge [1]	–	–	–	–	–	–
Luo <i>et al.</i> [2]	Y	–	–	–	–	–
Pinho <i>et al.</i> [3]	Y	–	–	–	–	–
Washburn <i>et al.</i> [4]	Y	–	–	Y	Y	–
Rokis and Kirikova [5]	Y	–	–	–	–	–
Zhang <i>et al.</i> [19]	Y	–	–	Y	Y	–
Martini <i>et al.</i> [16]	–	Y	–	–	Y	–
Strandberg [17]	–	Y	–	–	–	–
Jones and Miller [13]	Y	–	–	Y	Y	–
This work	Y	Y	Y	Y	Y	Y

T4 Prompt

You are a workflow automation expert. Implement a Saga pattern for order processing with two parallel sub-services: Inventory Reservation (POST /inventory/reserve) and Payment Authorization (POST /payment/authorize). The input is an order event with fields order_id, items (array), total_amount, and customer_id. Execute both sub-services concurrently using asyncio.gather. If both succeed, POST /order/confirm and return state 'confirmed'. If either fails, trigger compensating actions: POST /inventory/release for the inventory reservation and POST /payment/void for the payment authorization. Log all actions to 'saga.log'. Return the final state ('confirmed' or 'cancelled') and the compensation log. Handle exceptions at each step individually so that one failure does not prevent the other compensation from executing.

Example Generated Output (T1, AI)

The following is the Python script generated by GPT-4 Turbo for T1 (data aggregation pipeline). It is reproduced verbatim from the median-IC generation:

```
import pandas as pd
import json

df = pd.read_csv('input.csv')
# Remove rows with empty or non-numeric 'value'
df['value'] = pd.to_numeric(df['value'], errors='coerce')
df = df.dropna(subset=['value'])
# Group by category
result = df.groupby('category').apply(
    lambda x: x.to_dict(orient='records')
).to_dict()
with open('output.json', 'w') as f:
    json.dump(result, f, indent=2)
```

F Coordination Complexity Matrix

Table 12 quantifies the coordination properties of each task across five dimensions. Unlike the binary indicators in Table 5, this matrix uses exact counts to make the difficulty ordering quantitatively defensible.

Dimension definitions. *Branch Count* is the number of explicit conditional decision points. *Failure Paths* is the number of distinct failure-handling routes (retry branches, fallback paths,

Table 12: Coordination complexity matrix: exact counts across five dimensions for each task.

Task	Branch Count	Failure Paths	Parallel Paths	State Sync Points	Compensation Logic
T1 (Pipeline)	0	1	1	0	0
T2 (Retry)	1	3	1	0	0
T3 (Branch)	3	4	1	0	0
T4 (Orchestration)	0	4	2	2	2

error triggers). *Parallel Paths* is the number of concurrent execution threads. *State Sync Points* is the number of points where parallel paths must synchronize state. *Compensation Logic* is the number of compensating actions required on failure. The matrix makes the ordering concrete: T1 has no branching or synchronization; T2 adds one branch and three failure paths; T3 extends to three branches with four failure paths; T4 introduces parallel execution (2 paths), state synchronization (2 points), and compensation logic (2 actions). This ordering is defensible

because each successive task strictly dominates its predecessor on at least one dimension while equalling or exceeding it on all others.

G Evidence Synthesis

Table 13 provides a consolidated summary of the evidence for and against CDH and ACE across all task-paradigm combinations, without author-assessed strength labels.

Table 13: Summary of evidence for each finding across the benchmark.

Finding	Supportive Evidence	Non-Supportive Evidence
IC decreases with abstraction	AI $\mu(IC)$ = 34.5s vs. Python $\mu(IC)$ = 180.5s. Reduction stable across all four tasks (80.9%)	Python T3 IC (127s) lower than T2 (183s), showing IC not strictly monotonic with task difficulty for code
RC escalates for non-code paradigms	n8n 20.4× (T1: 5s → T4: 102s); AI 4.9× (T1: 18s → T4: 88s)	Python RC flat across all tasks (19–23s). RC escalation limited to coordination-heavy task (T4)
ACE: TWC ratio compresses	AI/Python ratio: 2.3× (T1) → 1.9× (T4). n8n/Python: 1.8× (T1) → 1.2× (T4)	Only four data points; no functional form established; compression driven primarily by T4
LCRI ordering: Python < n8n < AI	Consistent across all four tasks. Python: 19.7–35.2%; n8n: 31.2–52.3%; AI: 60.6–83.2%	n8n LCRI drops at T3 (31.2%), breaking monotonic increase observed at T1–T2
Non-monotonic effects	T2–T3 Python IC inversion, T3–T4 AI IC inversion, n8n T3 LCRI drop	Alternative interpretation: IC reflects task-specific features (dependency breadth, design pattern familiarity) rather than CDH
Verification cost invariant	VC ranges 14–31s across all cells; $\mu(VC) \approx 18$ –20s	Limited by standardized checklist; production verification (regression, security) may differ